



Achieve Peak Tensor Core Performance for GEMM on Blackwell via CUTLASS Python

Vicki Wang, Distinguished Software Engineer
Manas Sahni, Senior Software Engineer

GTC | March 2026

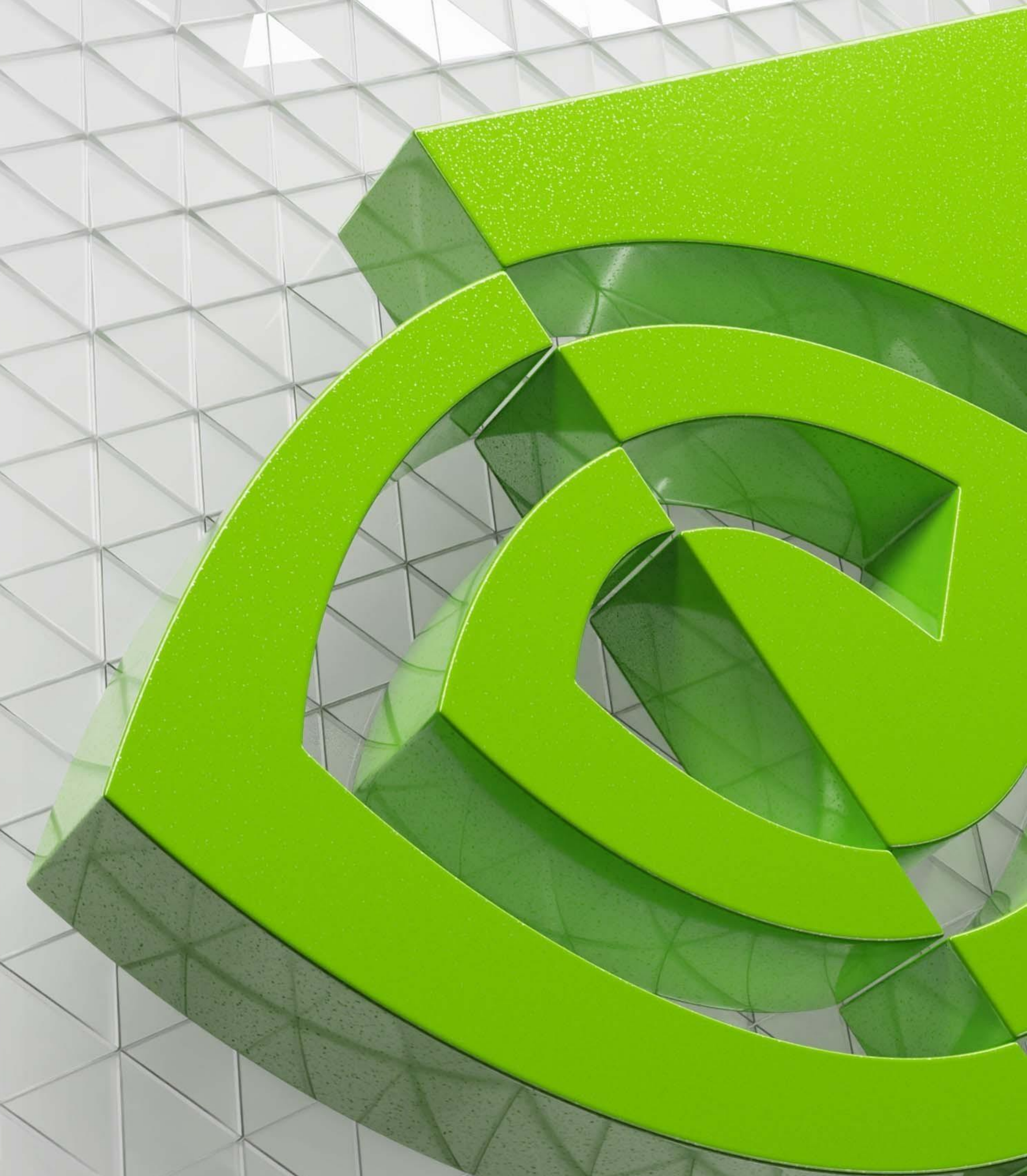
Agenda

Introduction

Progressive GEMM Optimization

CUTLASS DSLs

Python Operator API



CUTLASS

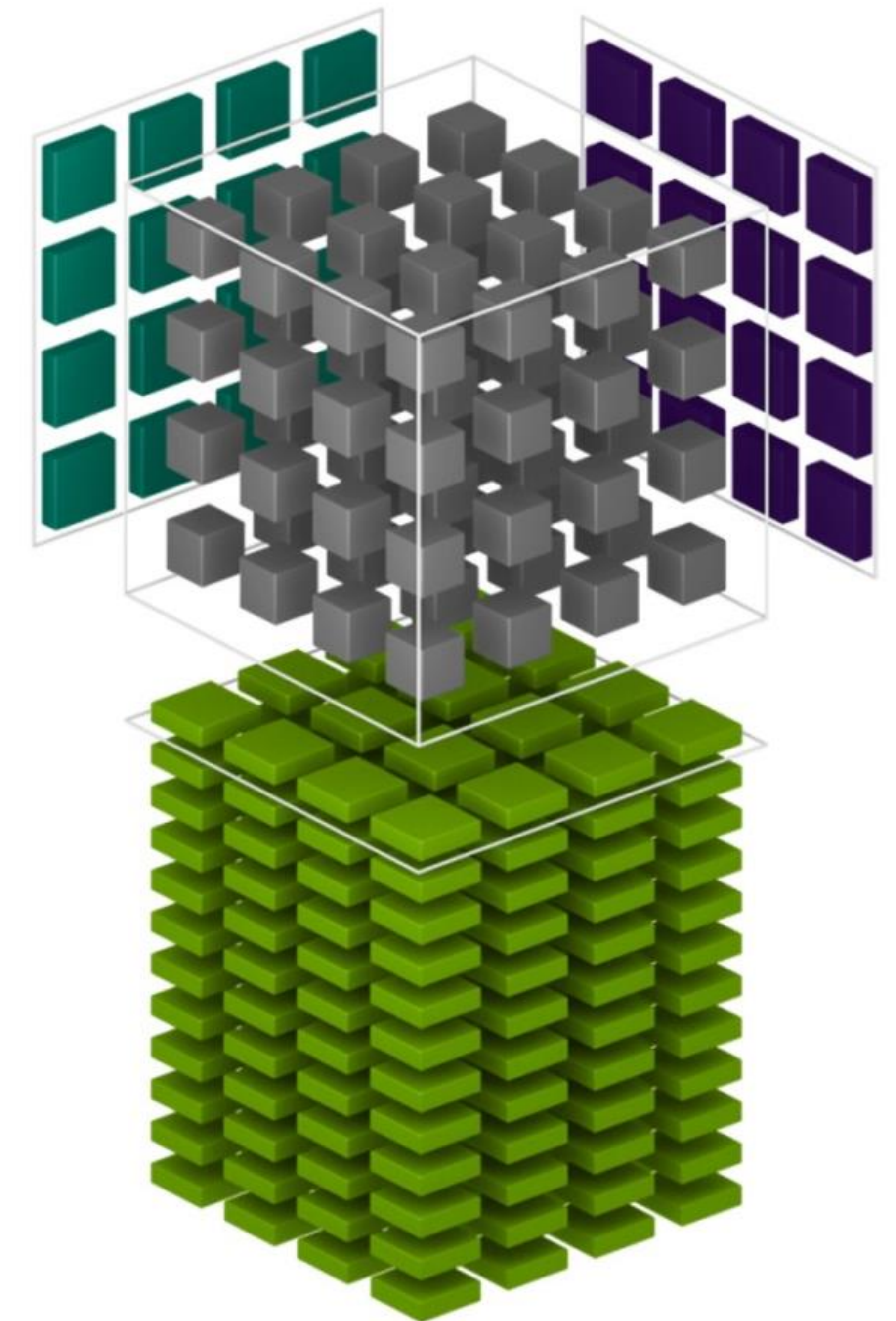
Programming Tensor Cores

Core Technology

- CUTLASS is a comprehensive library providing high-performance abstractions for:
 - **Matrix-Matrix Multiplication (GEMM)** - The foundation of AI and deep learning computations
 - **Convolution Operations** - Through implicit GEMM algorithms
 - **Mixed-Precision Computing** - Supporting FP64, FP32, TF32, FP16, BF16, FP8, FP6, FP4 and integer data types
 - **Tensor Core Programming** - Optimized for NVIDIA Ampere through Blackwell architectures

Dual Programming Interface

- **C++ Templates:** Production-grade performance with extensive customization
- **Python DSLs:** Performance + rapid prototyping, with 100x faster compile times



NVIDIA solution for *programming* Tensor Cores

Why CUTLASS Python?

Enabling innovations for SOL performance

To reach SOL perf:

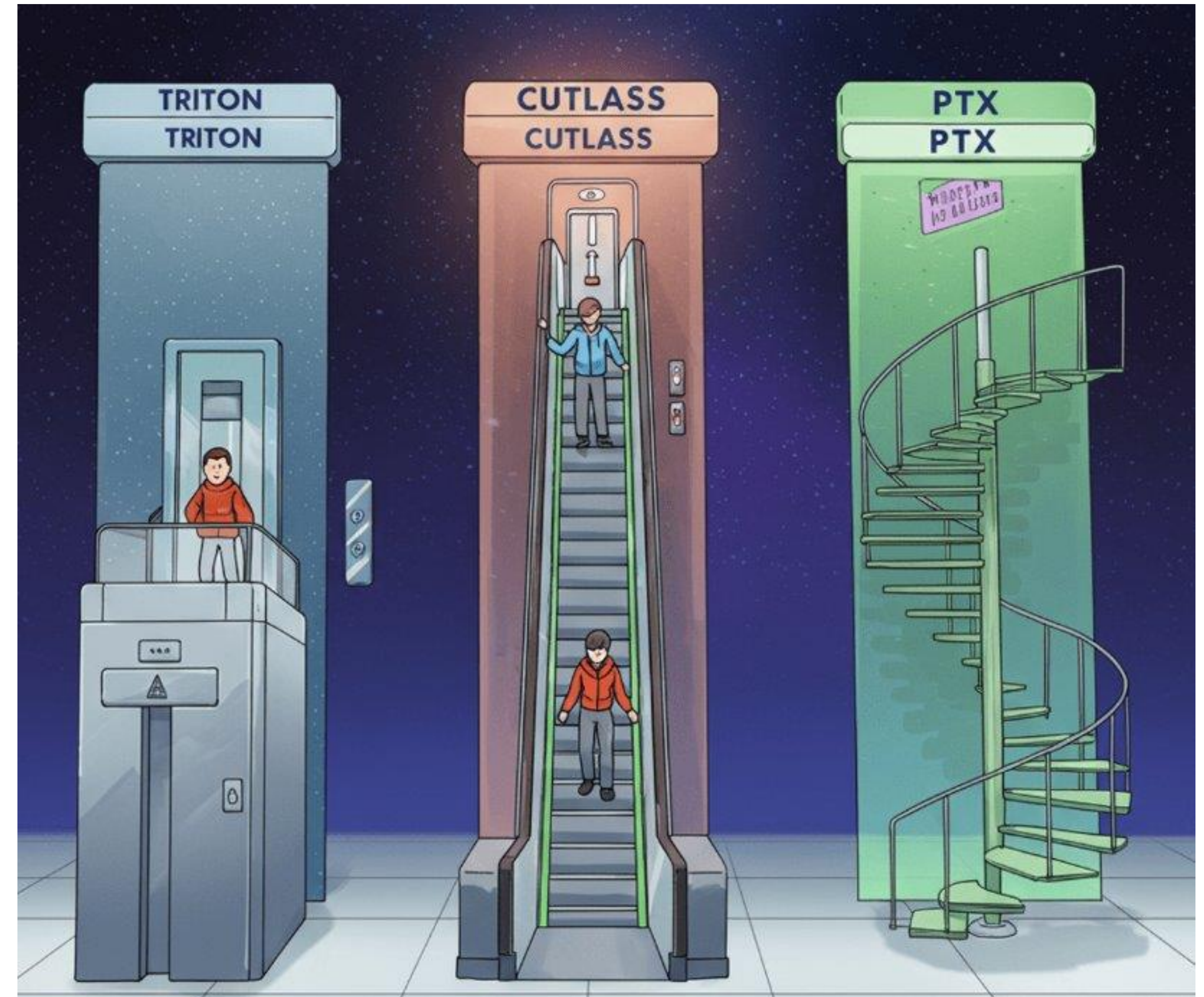
- **Strategies** require lower-level *abstractions*
- Advanced hardware features like **PDL** require fine grain control

Why CUTLASS:

- Available on day 0 with full control!
- Expressive abstractions for performance in all cases
- Modular and extensible design robust throughout GPU generations

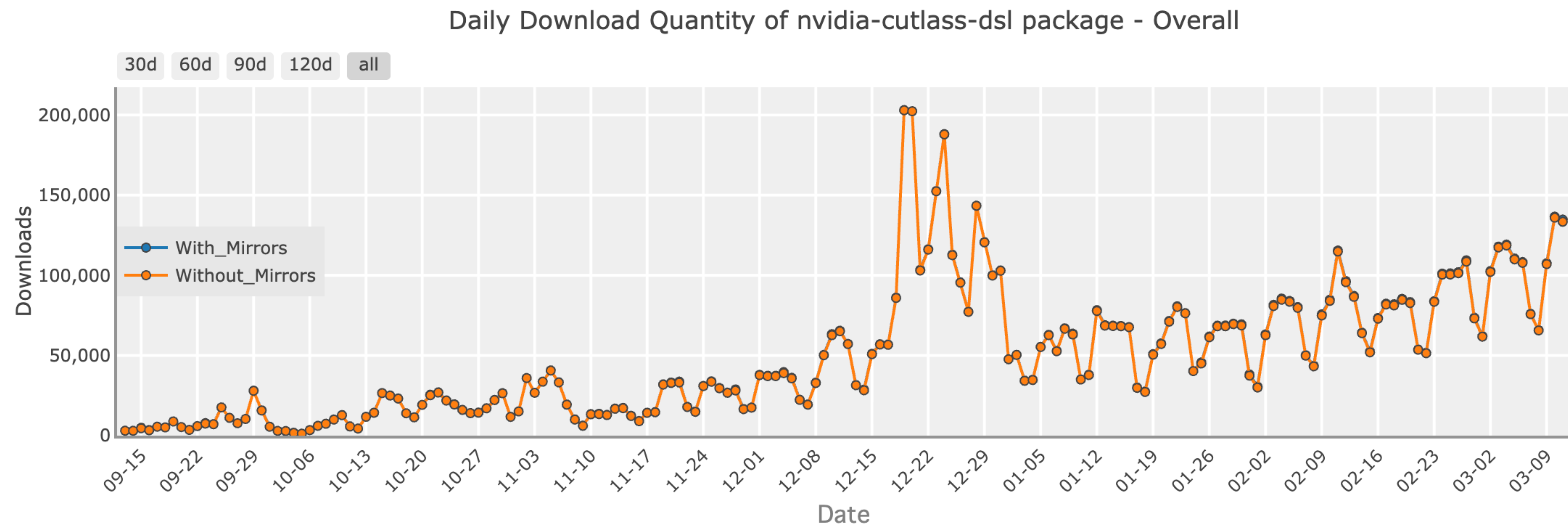
Why CUTLASS Python:

- C++ templates are inconvenient
- C++ suffer from slow compilation time
- The DL space fully embraces the Python ecosystem



CUTLASS Python Adoption Story

- 4.5 coming soon, will support fine grain compilation control, python linter support w/ mypy, and better source code correlation for profiling/debugging
- Peaked at **200K** downloads in a single day, and **~8.6M** downloads in total
- Used in FAv4, QuACK, vLLM, SGLang, and Sonic-MoE
- More SOL kernels (attention, MoE) are integrated into TRTLLM and FlashInfer
- Enable developers to be more efficient in authoring optimized kernels on GPU



CUTLASS 4.4 Stack

More pre-tuned recipes



More Control

Device Layer (Gemm Universal, Conv, etc.)

CUTLASS Kernel Layer

SM90
GEMM TMA
Warp Specialized
Ping-pong

SM90
GEMM TMA
Warp Specialized
Cooperative

SM100
GEMM TMA
Warp Specialized

SM100
GEMM TMA
A from TMEM

....

Collective Layer

Collective
MMA
SM90 TMA SS

Collective
MMA
SM100 TMA

Collective
MMA
SM100
ATMEM

Collective
Epilogue TMA

....

CuTe
Tiled Copy & Tiled MMA

CuTe Atoms

TMEM load

TMEM store

TMA

MMA SM90

MMA SM100

....

CuTe DSL



Current Python offering
focused here

Agenda

Introduction

Progressive GEMM Optimization

CUTLASS DSLs

Python Operator API



Why Do We Need Tutorial GEMMs?

CUTLASS Python provides plenty of examples!

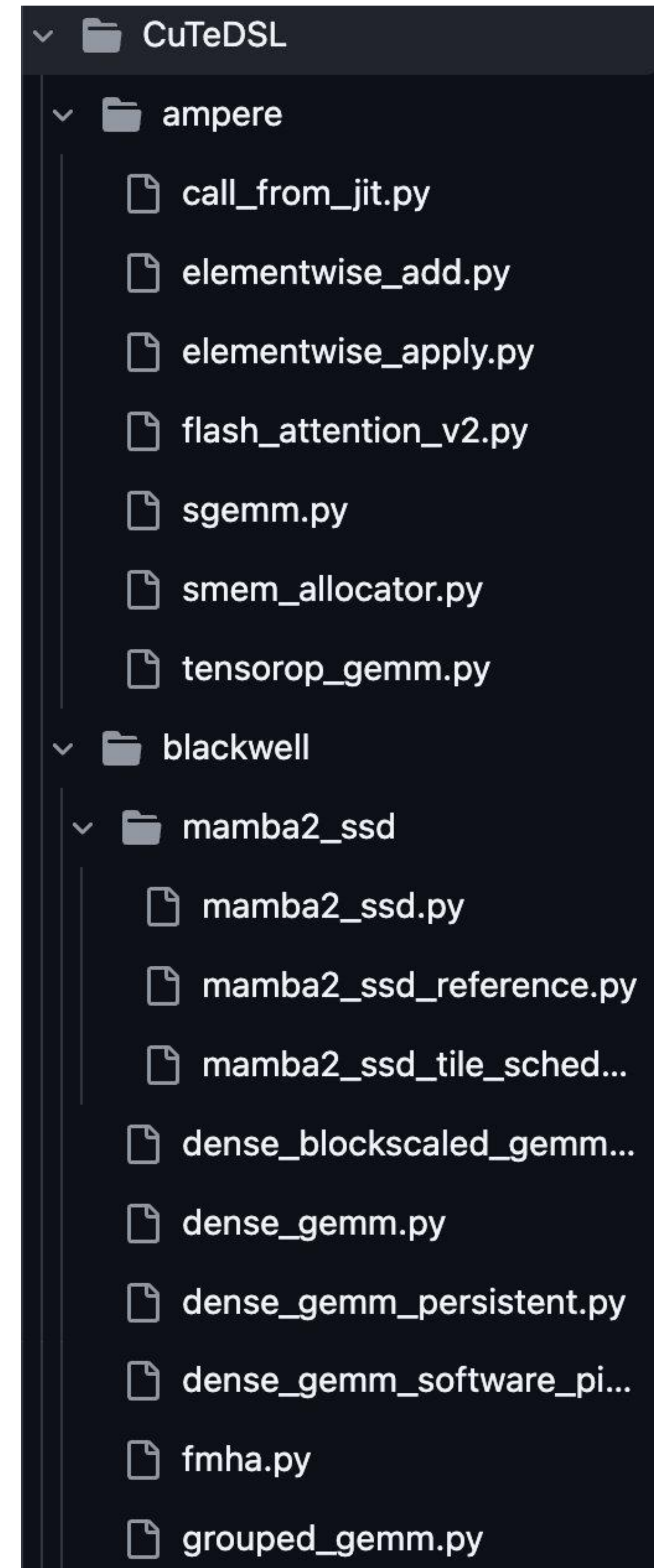
- Each of them combines multiple optimizations for peak performance

Beginners may struggle to discover the key path that leads to SOL 😞

Finding the most relevant kernel requires understanding...

- What are the key primitives for a given HW feature?
- How and when to use them to address performance issues?

Where do you start?



GEMM Algorithm

Characteristics

Parallelizable

- Each $C_{i,j}$ can be computed independently
- Allows full utilization of SMs

Data-reusable

- The row i of C uses the same data from A_i
- The column j of C uses the same data from B_j
- Opens the possibility to reduce I/O pressure

Block-friendly

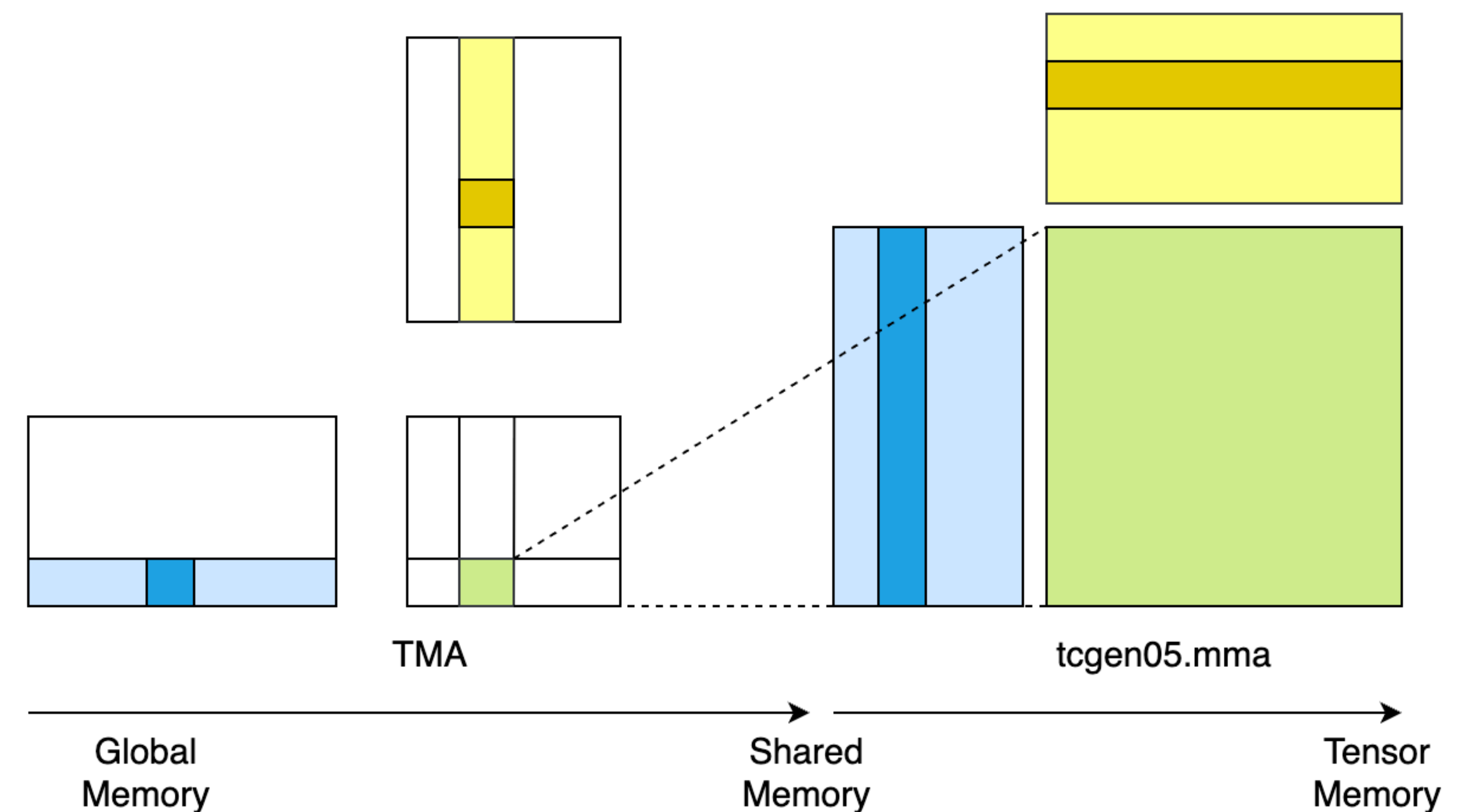
- Blocks of elements can be processed together
- Helps reduce I/O pressure for each SM
- Allows the use of MMA instructions

Bottleneck-flexible

- The bottleneck is **varied** for different problem sizes
- To reach SOL performance, need to match **strategies** according to problem sizes

Testing configuration on B200:

- Using random float as inputs
- Testing with floating clock
- Using nsight-compute to capture the execution time
- Power at 1000W



What's Covered in This Tutorial?

Optimizing GEMM step-by-step in 300 lines of Python

- This tutorial showcases how to:
 - Use Blackwell features via CuTe DSL
 - Analyze and improve the performance for different problem sizes



Code:
bit.ly/cute-tutorial-gemm

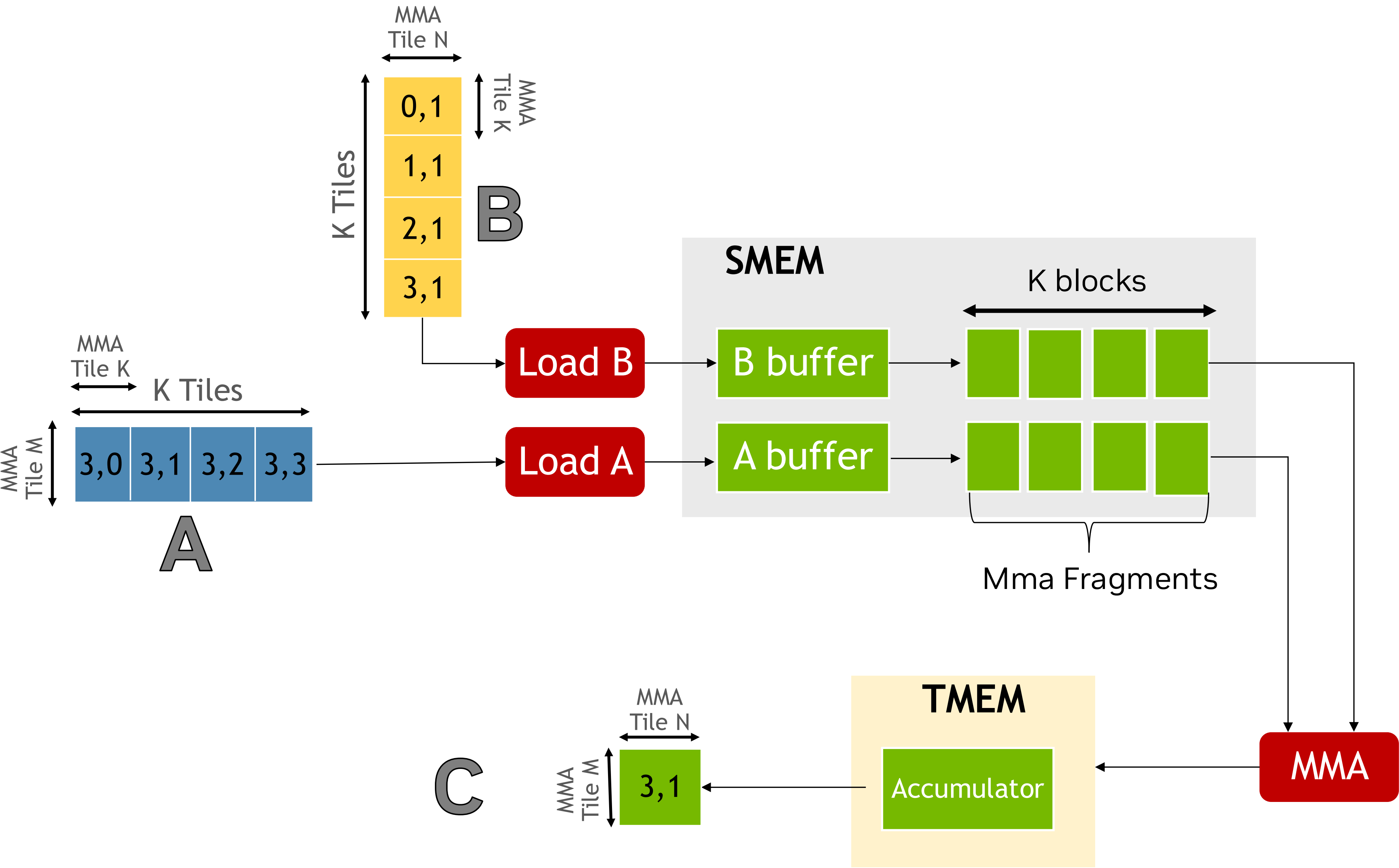
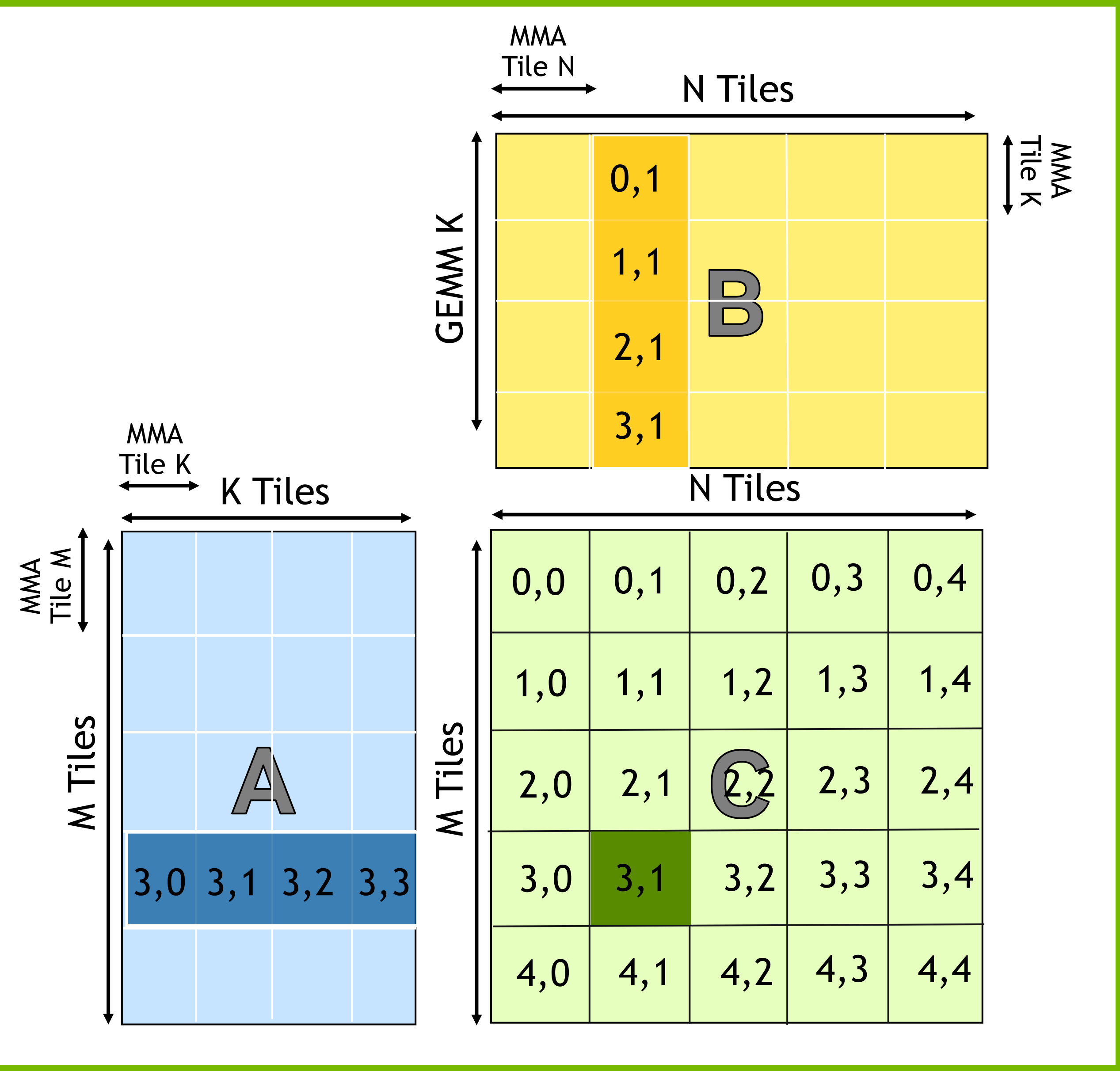
Strategies Used		Cumulative Benefits
Today	Software-pipelined GEMM	Baseline implementation
	+ 2 CTA optimization	Improved latency hiding
	+ Warp-specialization and TMA store	Better epilogue phase overlap
	+ Static Scheduling persistency	Pipeline bubbles hidden among waves
	+ Preferred and fallback clusters	SMs occupancy improved
Github	+ TMA L2 prefetch	DRAM latency hidden
	+ CLC dynamic scheduling	Better workload balance for multi streams
	+ PDL	Kernel launch bubbles hidden

Performance

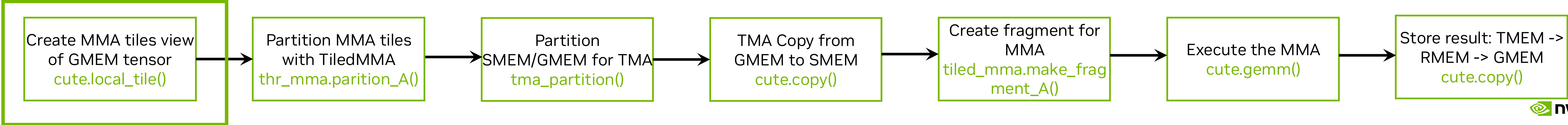
A large, thick green arrow pointing downwards, positioned to the right of the table, indicating that the cumulative benefits lead to improved performance.

Tutorial 0: Blackwell MMA GEMM

Workflow of the basic GEMM

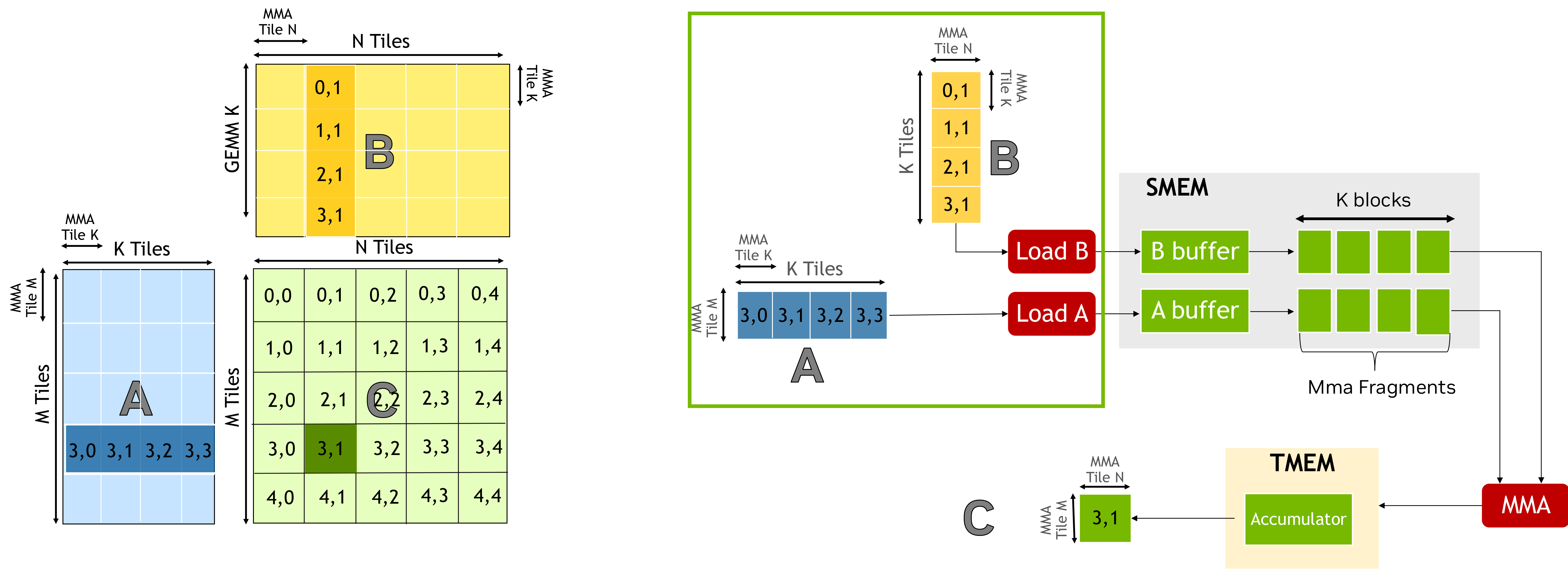


The overall procedures:

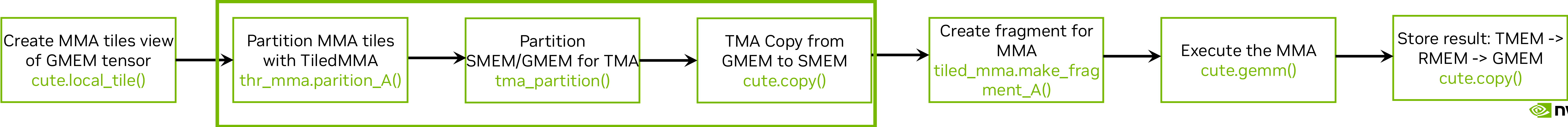


Tutorial 0: Blackwell MMA GEMM

Workflow of the basic GEMM

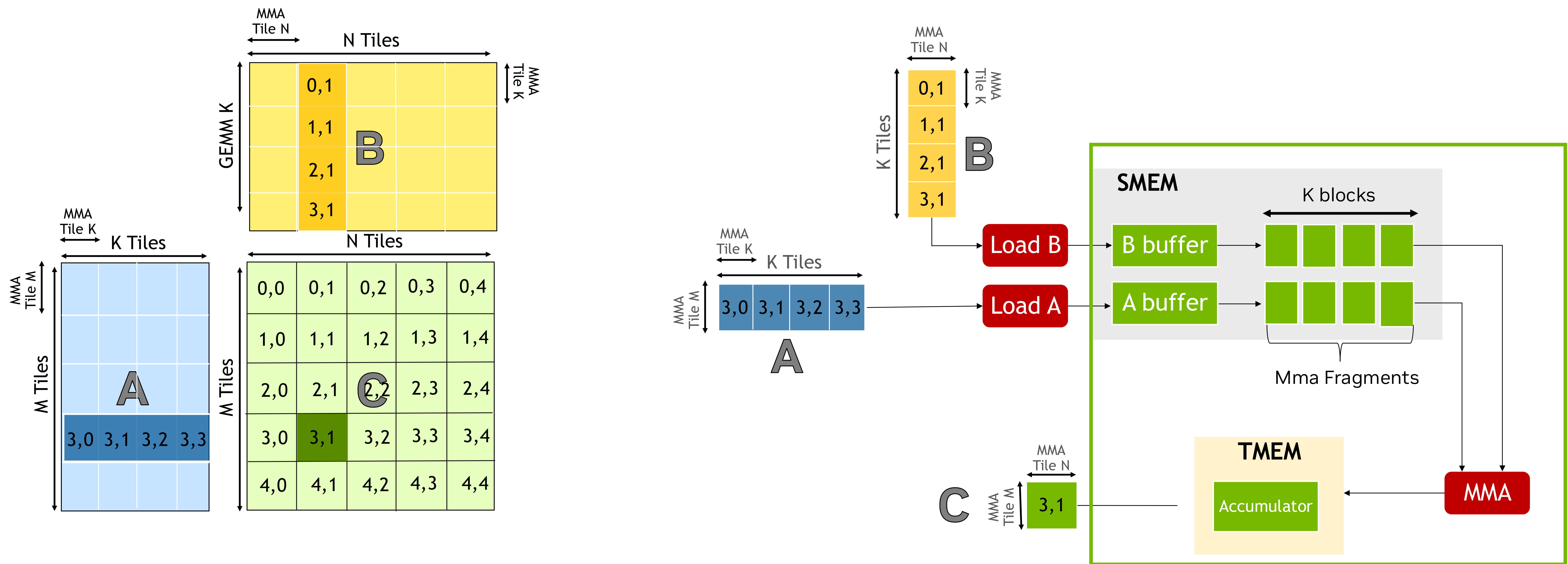


The overall procedures:

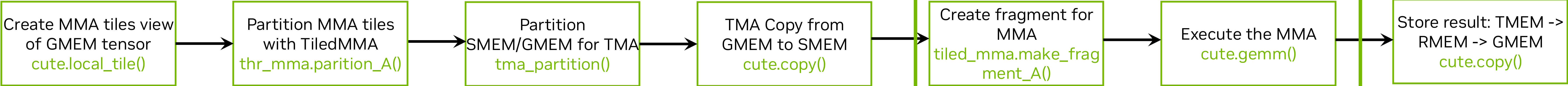


Tutorial 0: Blackwell MMA GEMM

Workflow of the basic GEMM

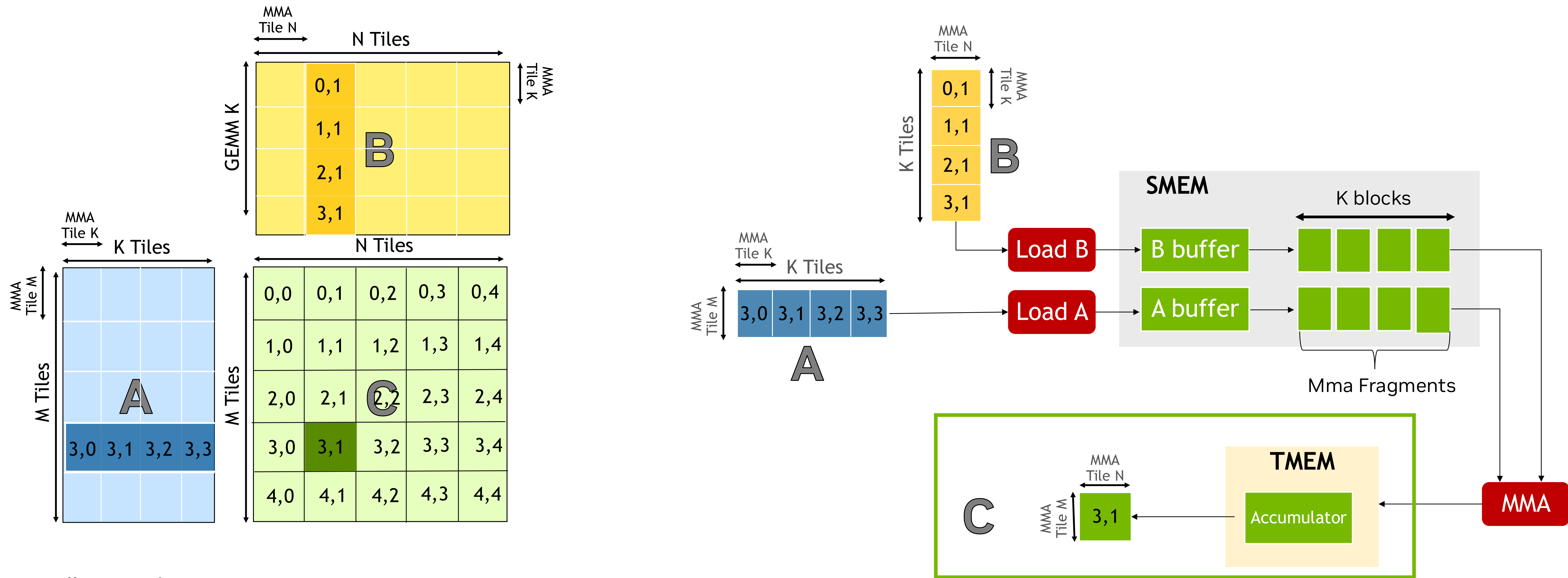


The overall procedures:

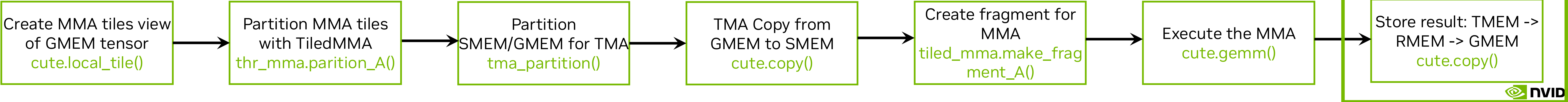


Tutorial 0: Blackwell MMA GEMM

Workflow of the basic GEMM



The overall procedures:



Tutorial 0: Software Pipeline

CTA tile size 128(M)x256(N)x64(K)

Hide the DRAM load latency

- Use multiple ab buffers for TMA load
- `ab_stages` **limited by SMEM capacity**

Prefetch a few stages of TMA load

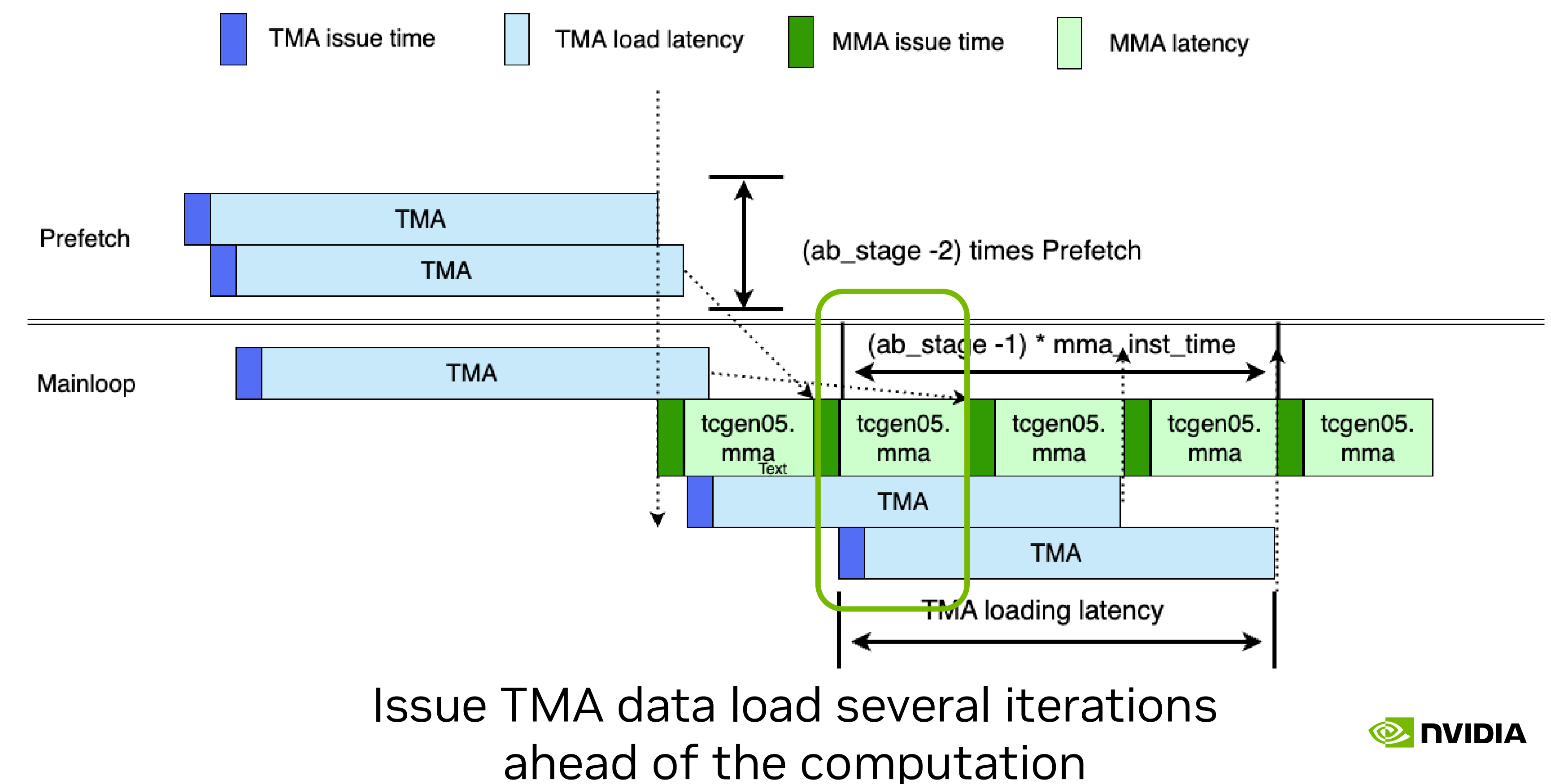
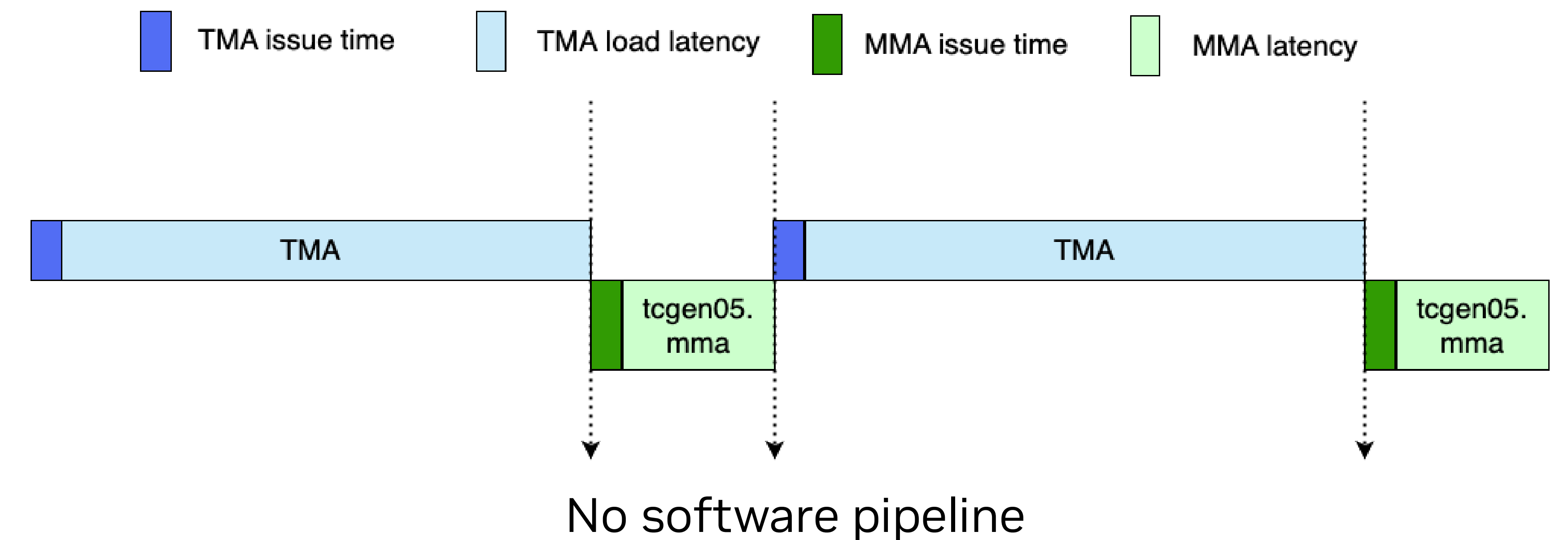
- The latency hiding ability:

$$(\text{ab_stages} - 1) * \text{one_stages_mma_inst_time}$$

```
for k_tile_idx in cutlass.range(
    num_k_tiles, prefetch_stages=ab_stages - 2
):
    # Issue TMA loads
    ab_empty = ab_producer.acquire_and_advance()
    # Load data for k_tile_idx + 2
    cute.copy(...)

    # Execute one K-block worth of MMA instructions
    ab_full = ab_consumer.wait_and_advance()
    # Compute for k_tile_idx
    cute.gemm(...)

    # Signal that the A/B buffers have been
    # consumed and are ready for the next load
    ab_full.release()
```

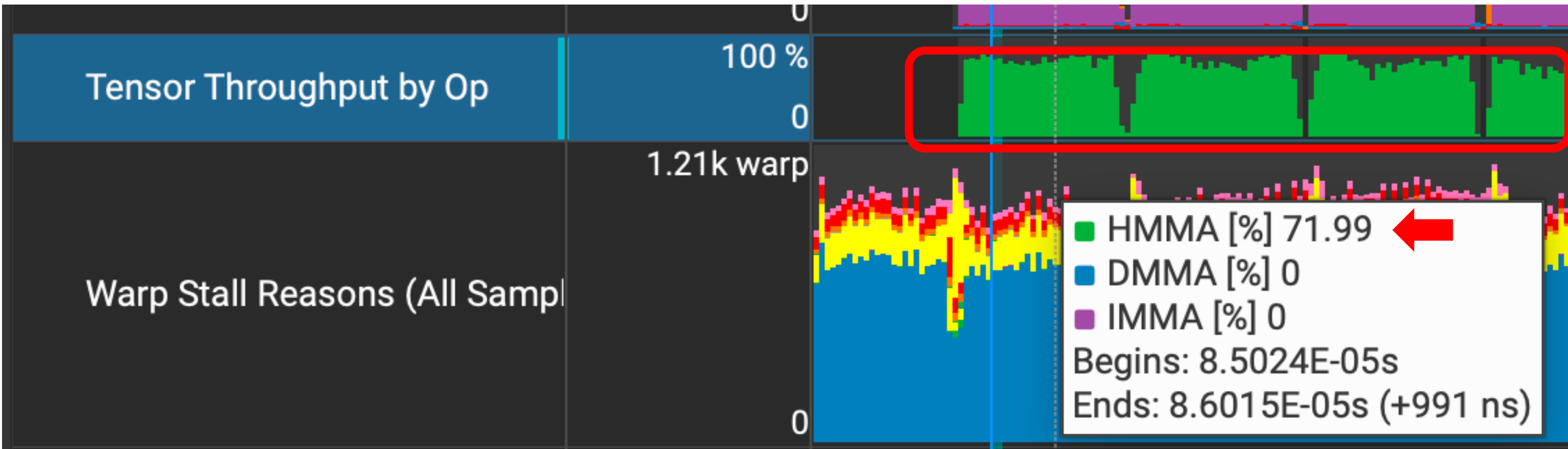


Tutorial 1: Motivation

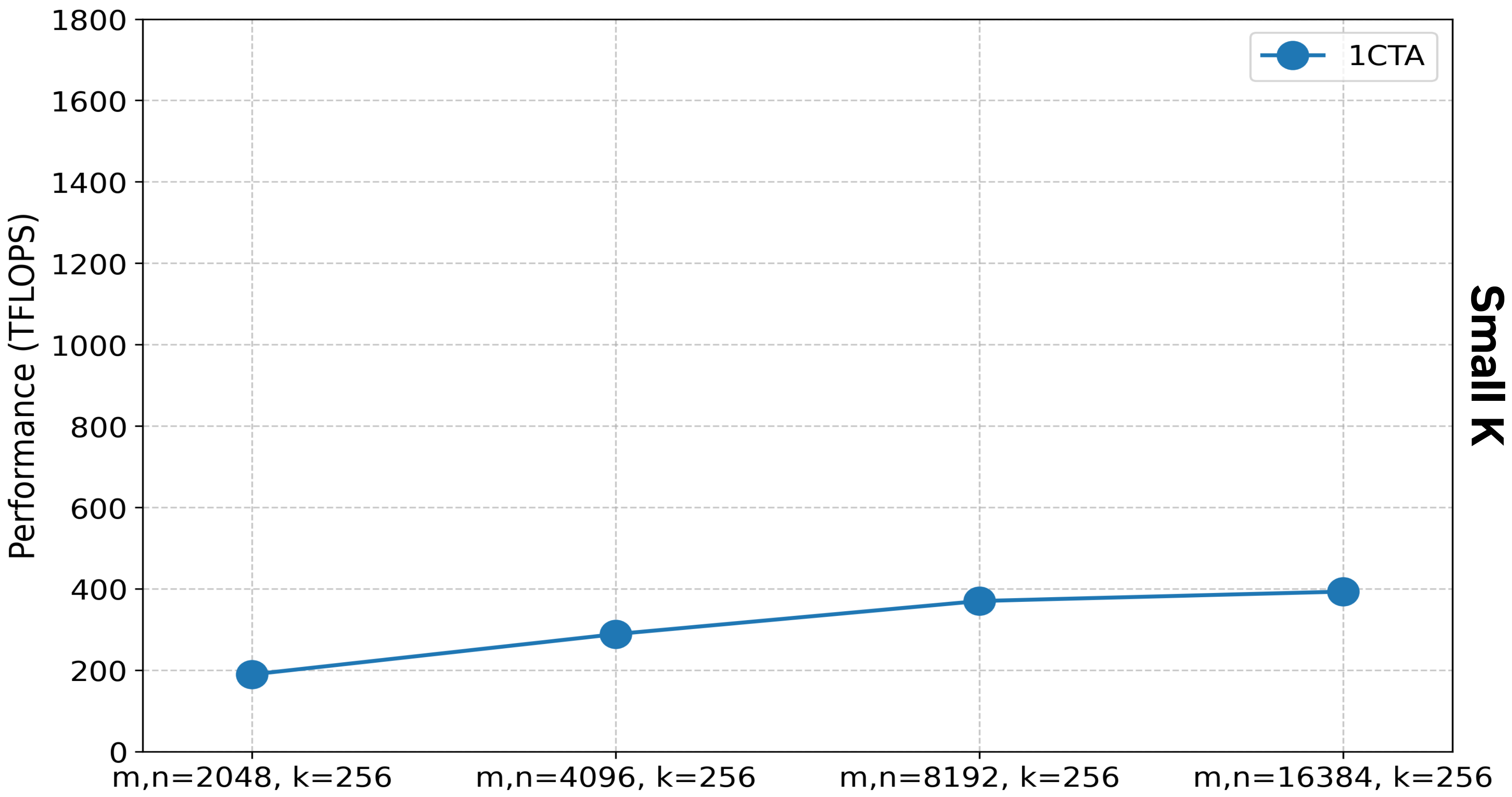
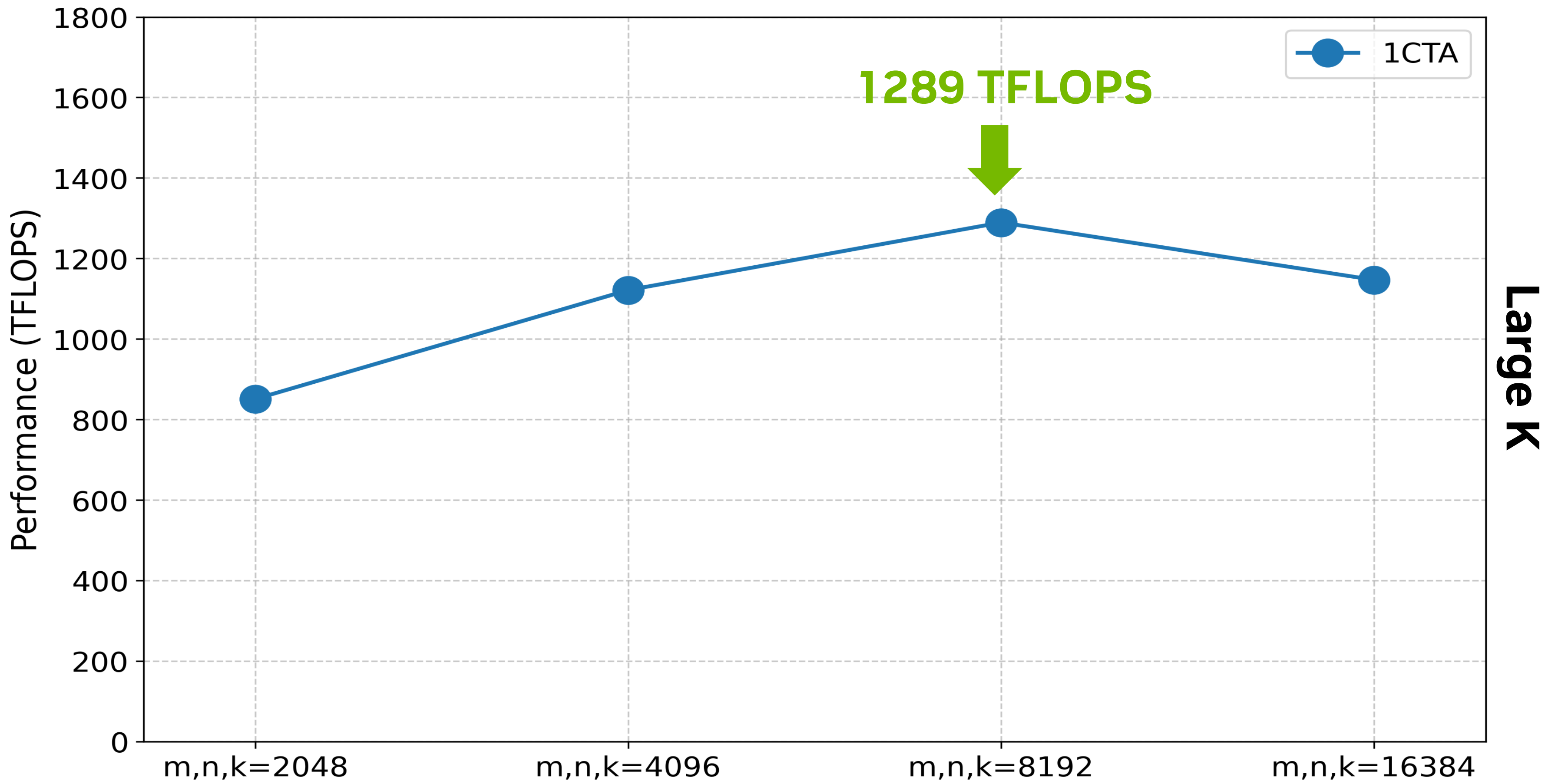
CTA tile size 128(M)x256(N)x64(K)

Known limitations:

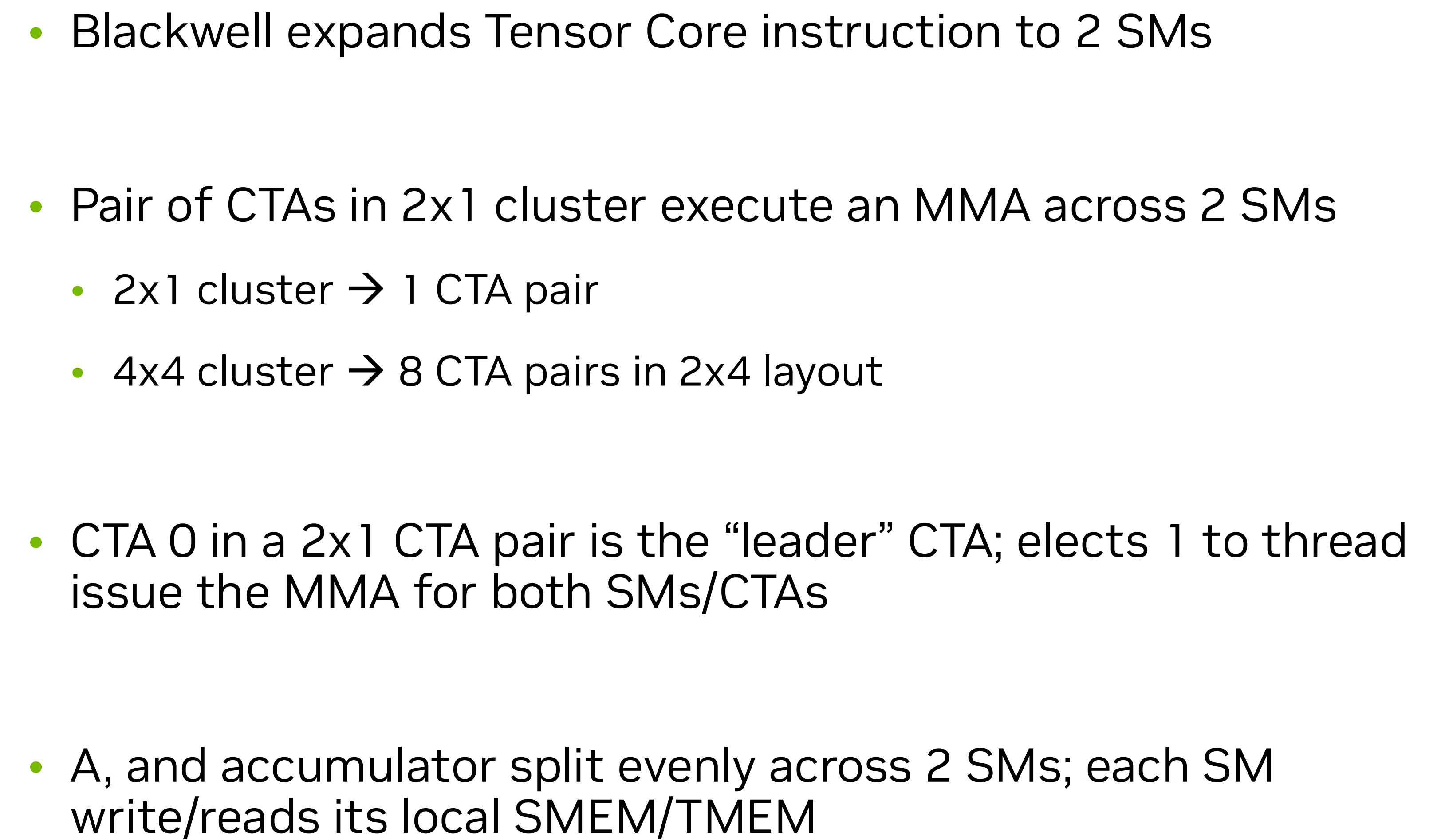
- Pipeline depth limited to 4 stages with 128x256x64 tile size
- Provide roughly $(4 - 1) * 512 = \mathbf{1536}$ cycles latency hiding
- Empirical benchmark shows latency can reach **1800** cycles
- Since **1536** < **1800** cycles, the data loading latency can not be fully hidden
- Compute throughput within a wave is only around 72%



GEMM Performance on B200



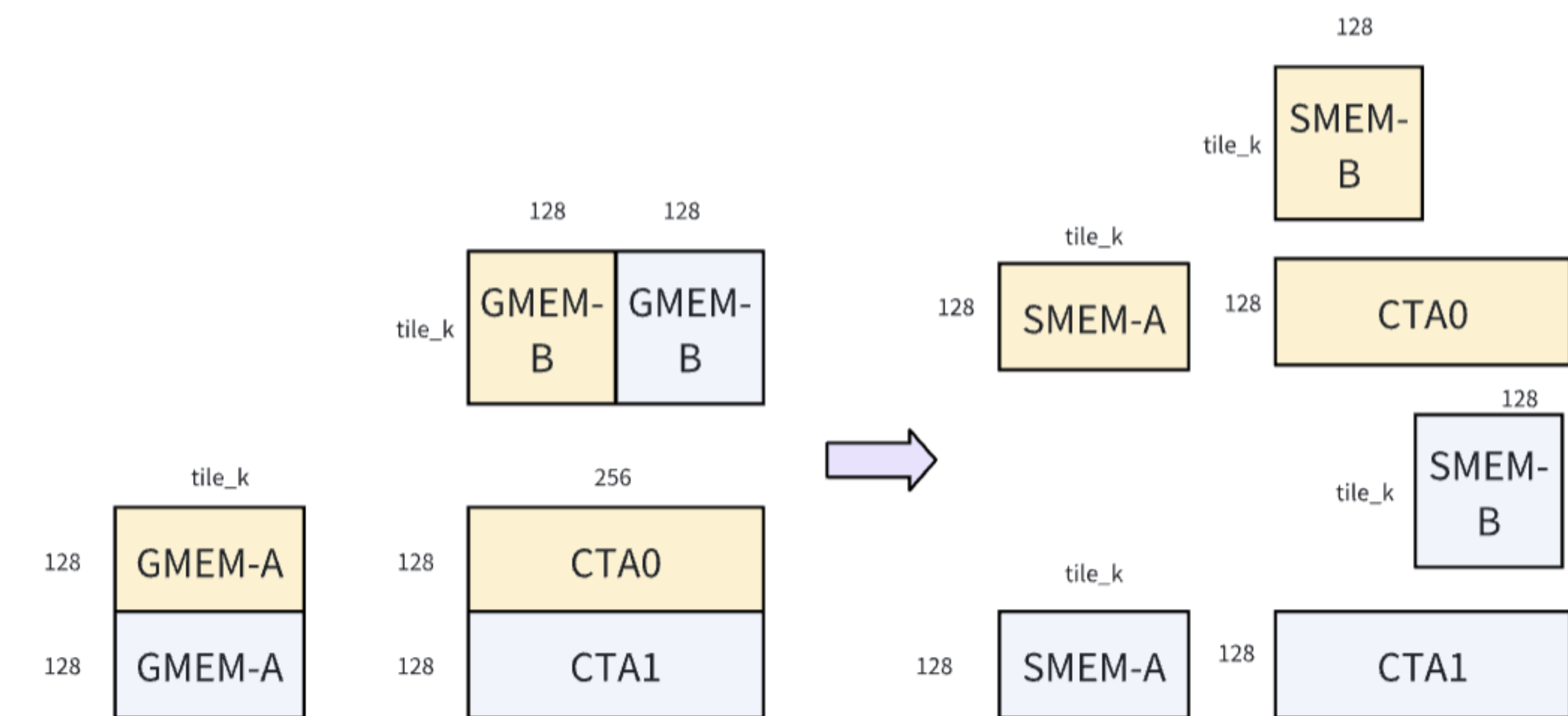
CTA tile size 128(M)x256(N)x64(K)



Tutorial 1: Enabling 2CTA

CTA tile size 128(M)x256(N)x64(K)

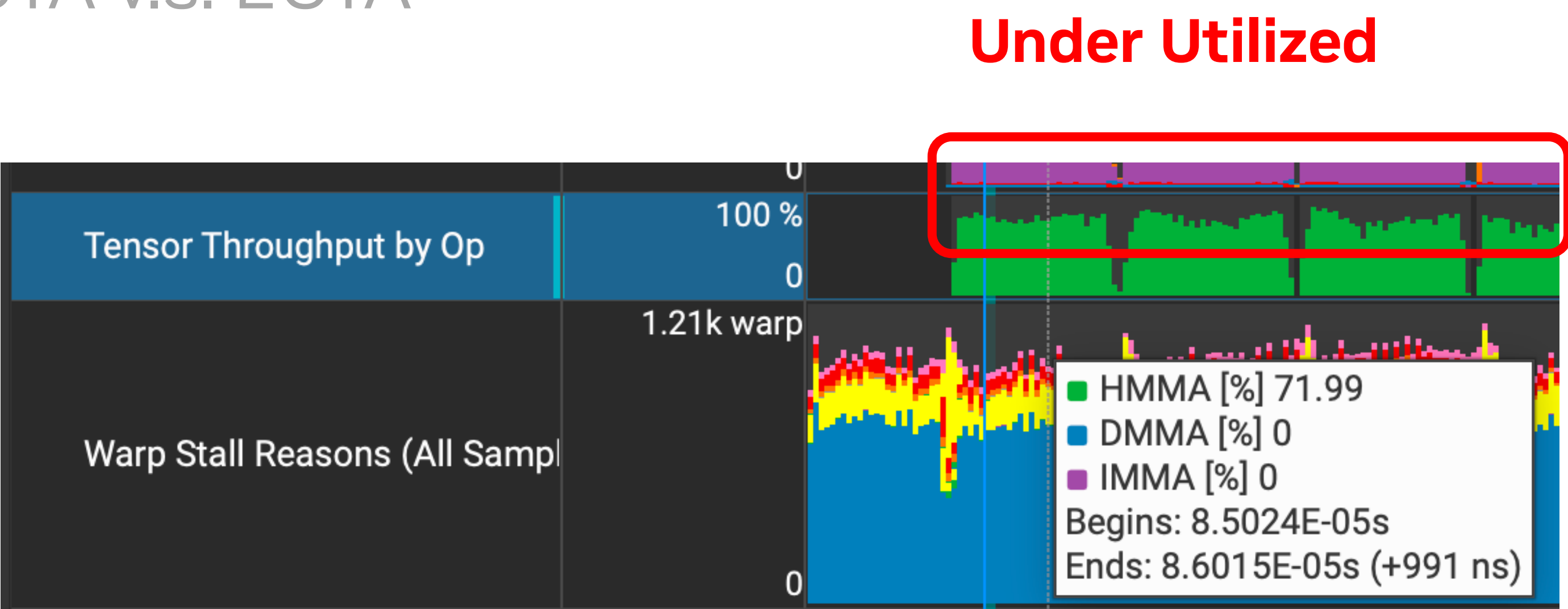
- 2SMs from a same TPC is paired together to process a larger tile size:
 - `mma_tiler_mnk` = **128**x256x64 → **256**x256x64
 - `cta_tiler_mnk` = 128x256x64
 - `Cluster_shape_mnk` = (1,1,1) → (2, 1, 1)
- Tiles mapping:
 - B is shared among the 2SMs
 - The SMEM space for B per SM **could be reduced by half**
- `ab_stages` size could be improved from **4** to **7**



	1 CTA SMEM usage (KB)	2 CTA SMEM usage (KB)
one_stage_A	16	16
one_stage_B	32	16
one_stage_all	48	32
num_ab_stages	227 // 48 = 4	227 // 32 = 7

Tutorial 1: Performance Status

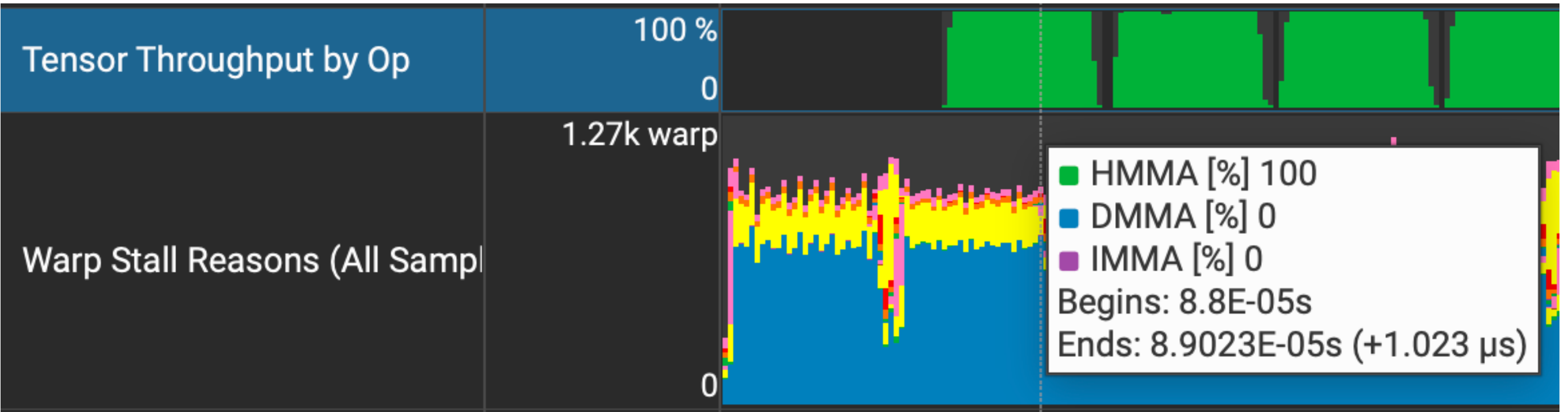
1CTA v.s. 2CTA



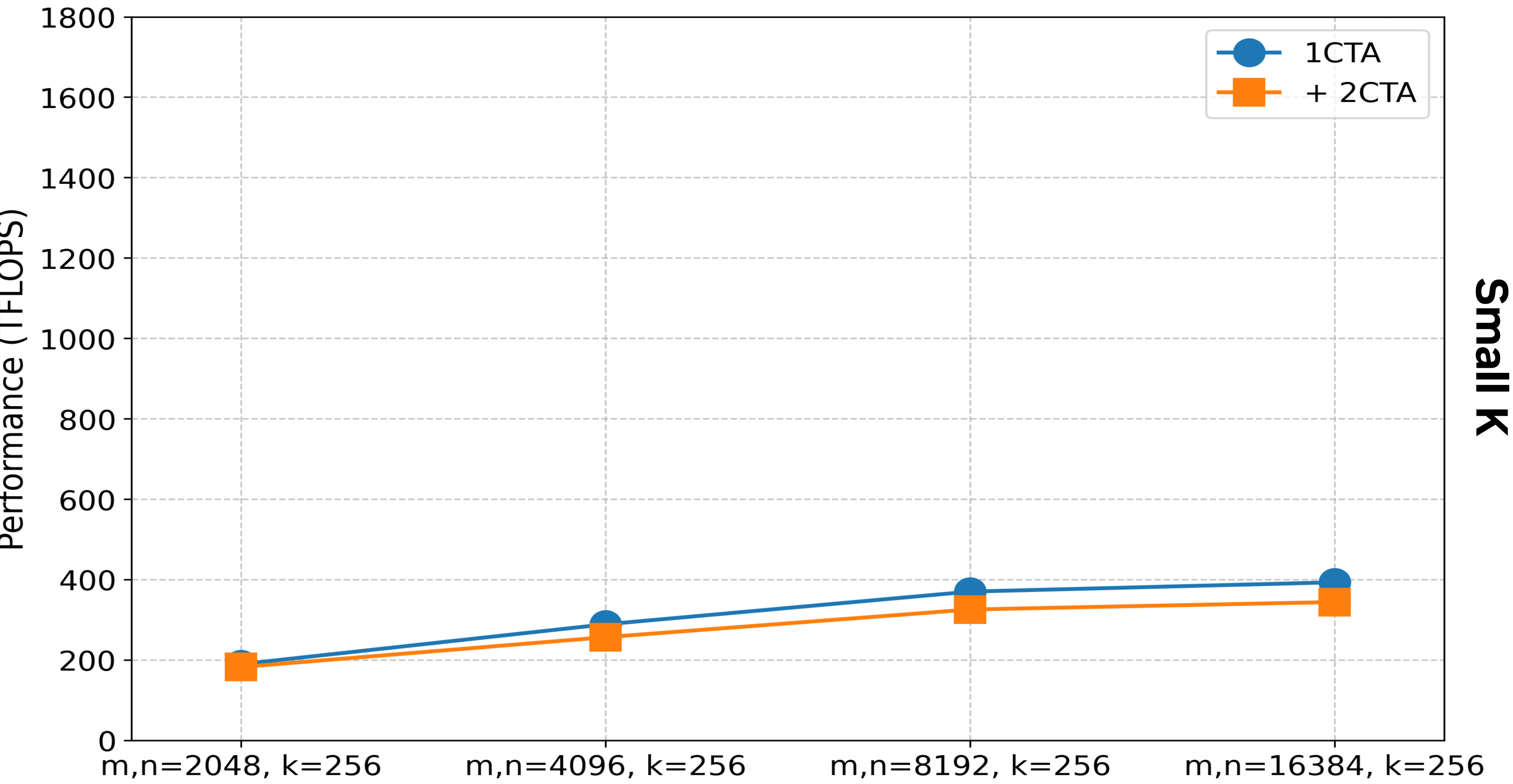
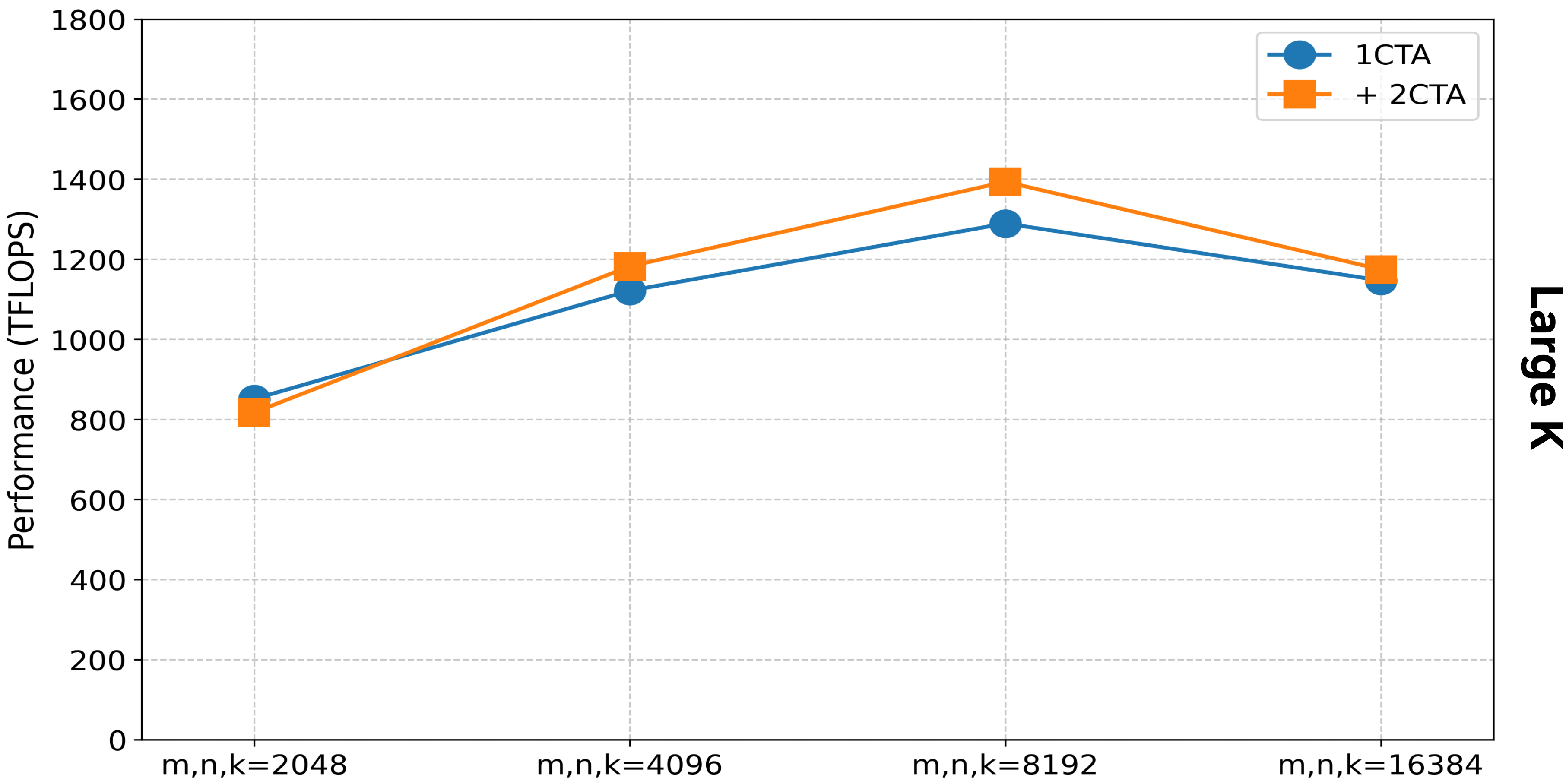
1CTA → 2CTA



Better Utilization!



GEMM Performance on B200

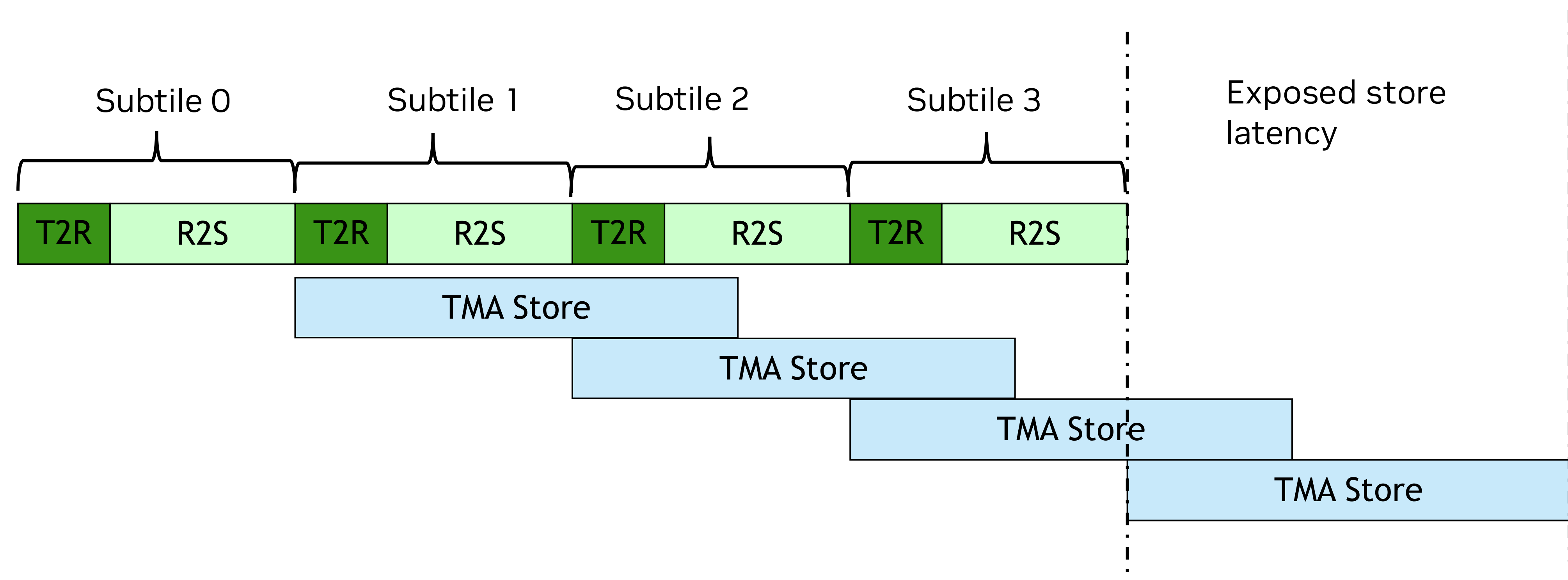


Tutorial 2: Epilogue Optimization and Warp-Specialization

Epilogue Optimization - Using TMA Store

- In Tutorial 2, we change to use TMA Store for **better coalesced store**

- Step 1: TMEM -> RMEM
- Step 2: RMEM -> SMEM
- Step 3: SMEM -> GMEM
- Sub-tiling for R2S and TMA Store overlap to reduce SMEM and register usage:



```
# Epilogue tiling loop
for subtile_idx in cutlass.range(subtile_cnt):
    # Slice
    ...
    # TMEM -> RMEM
    cute.copy(tiled_copy_t2r, tCtC, tCrC)

    # type conversion
    tCrC_out.store(tCrC.load().to(io_dtype))

    # RMEM -> SMEM
    cute.copy(tiled_copy_r2s, tCrC_out, tCsC)

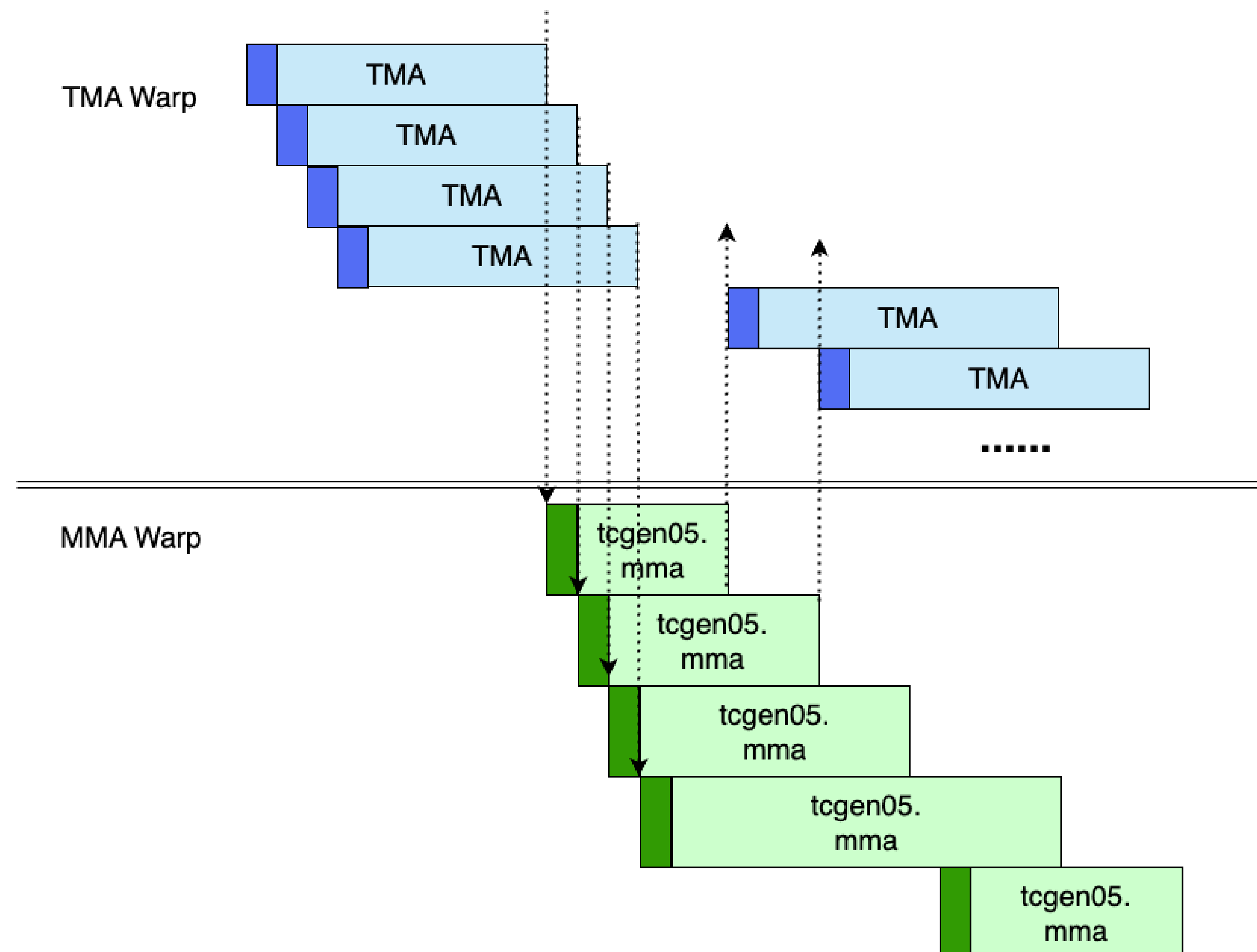
    # Memory fence and barrier to ensure shared
    # memory stores are visible to TMA stores
    cute.arch.fence_view_async_shared()
    epilogue_sync_barrier.arrive_and_wait()

    # SMEM -> GMEM
    if warp_idx == epilogue_warp_ids[0]:
        cute.copy(tma_c.atom, tCsC, tCgC)
```

Tutorial 2: Epilogue Optimization and Warp-Specialization

Warp specialization optimization

- Warp Specialization can achieve similar data loading latency hiding as software pipeline optimization but provide **better instruction issue efficiency** as different instructions are distributed to different warps



```
# Warps ids for each part

epilogue_warp_ids = (0, 1, 2, 3)
mma_warp_id = 4
tma_warp_id = 5
threads_per_cta = 32 * \
    len((mma_warp_id, tma_warp_id,
        *epilogue_warp_ids))
epilog_threads = 32 * len(epilogue_warp_ids)

# Execute data copy
if warp_idx == tma_warp_id:
    ...

# Execute math computation
if warp_idx == mma_warp_id:
    ...

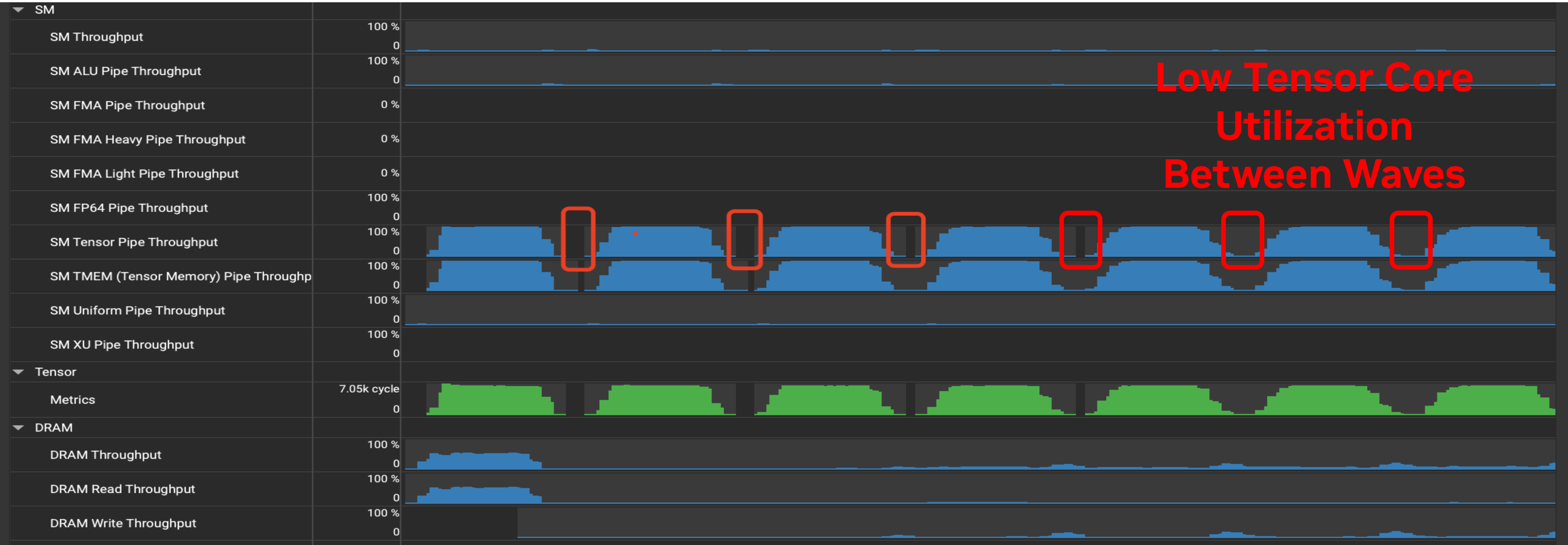
# Execute epilogue, store results to GMEM
if warp_idx < epilogue_warp_ids:
    ...

return
```

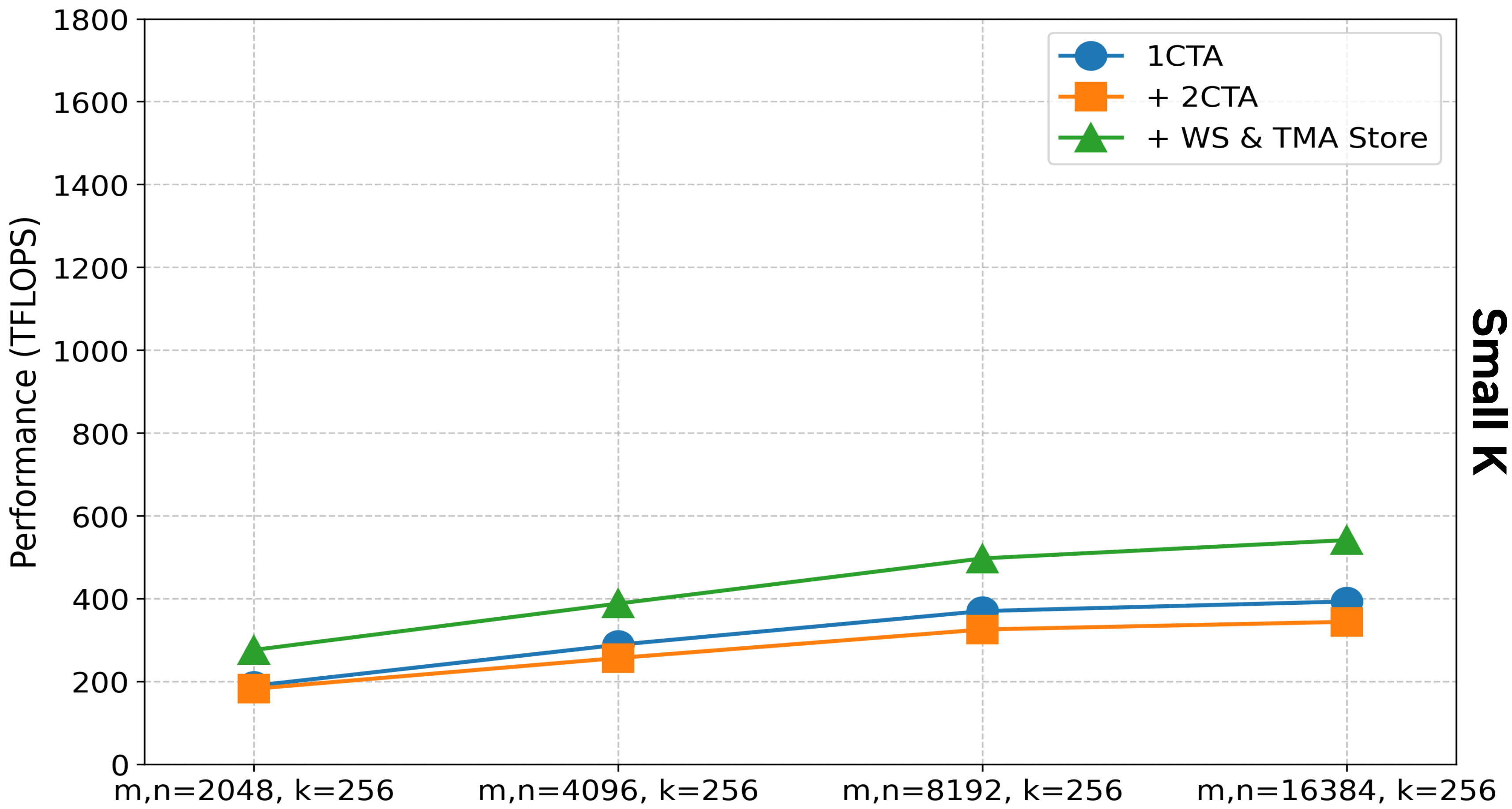
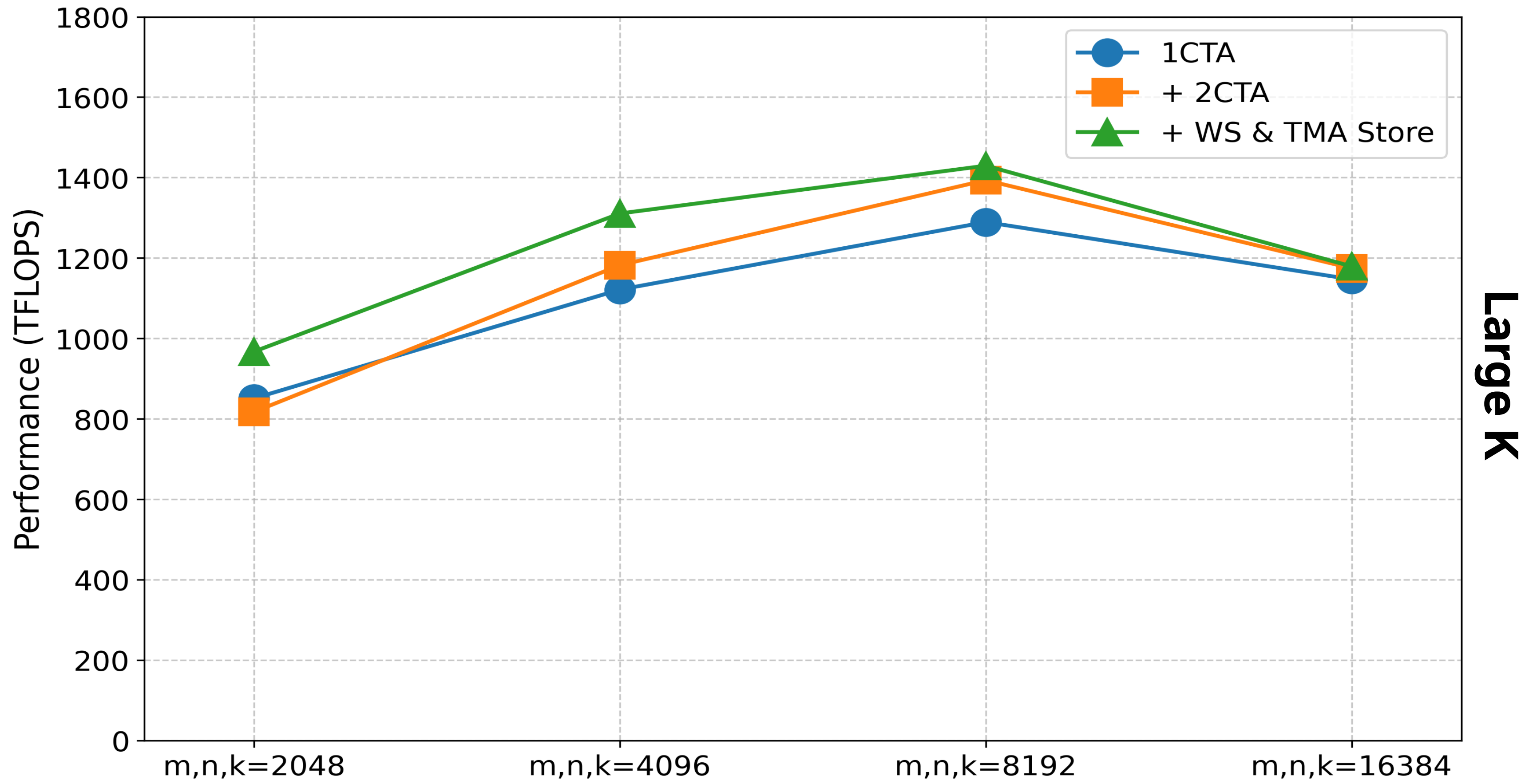
Tutorial 2: Performance Status

Warp specialization and epilogue optimization

- Optimization within a wave almost always beneficial...
 - For large K, main loop accounts for a very large proportion, so the speedup is not obvious
 - Small K cases always benefit
- But there is poor overlap among waves



GEMM Performance on B200

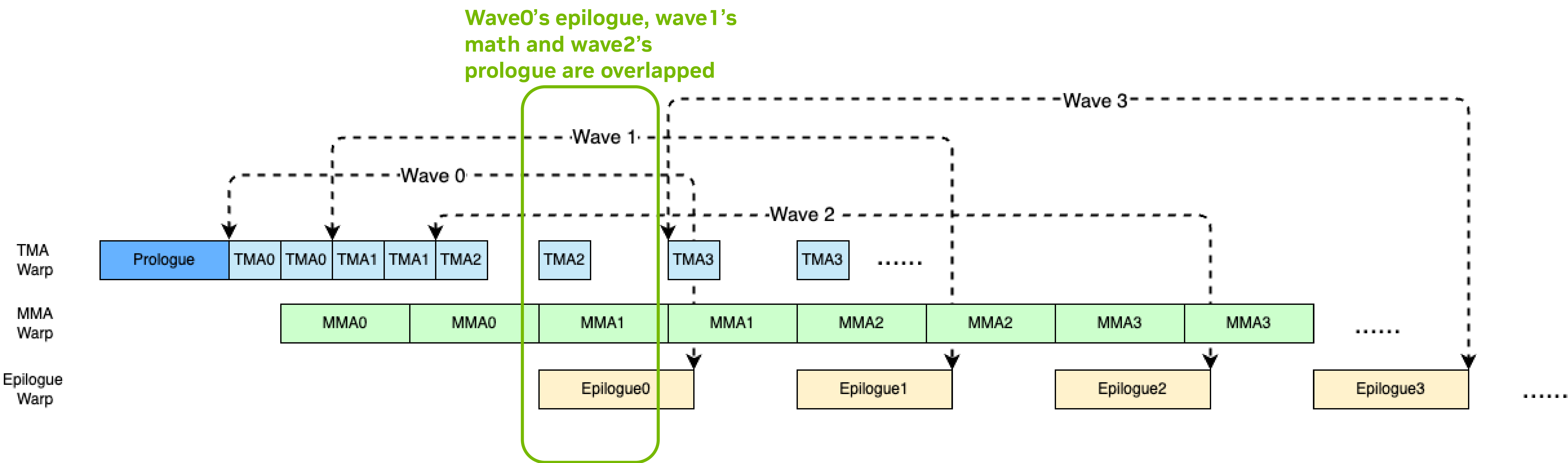


Tutorial 3: Phase Overlapping Optimization

Static persistent scheduler

Enable Persistent thread optimization

- Save (#waves-1) prologue, only need one prologue at the beginning
- The epilogue of wave $i - 1$ could be overlapped with mma of wave i

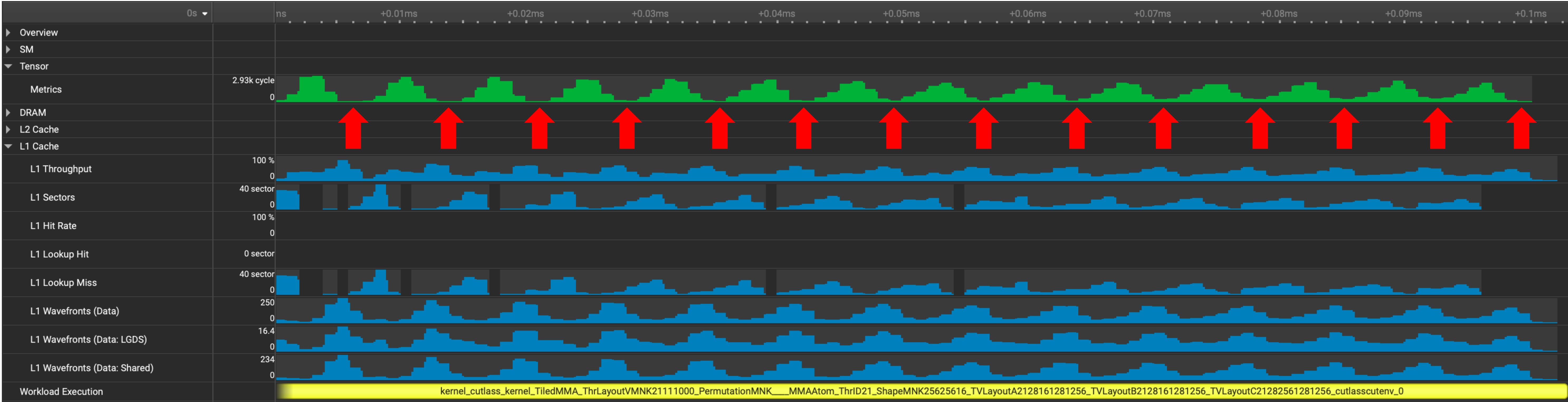


The execution timeline of warp specialize and persistent style GEMM kernel (assume perfect overlapping)

Tutorial 3: Performance Status

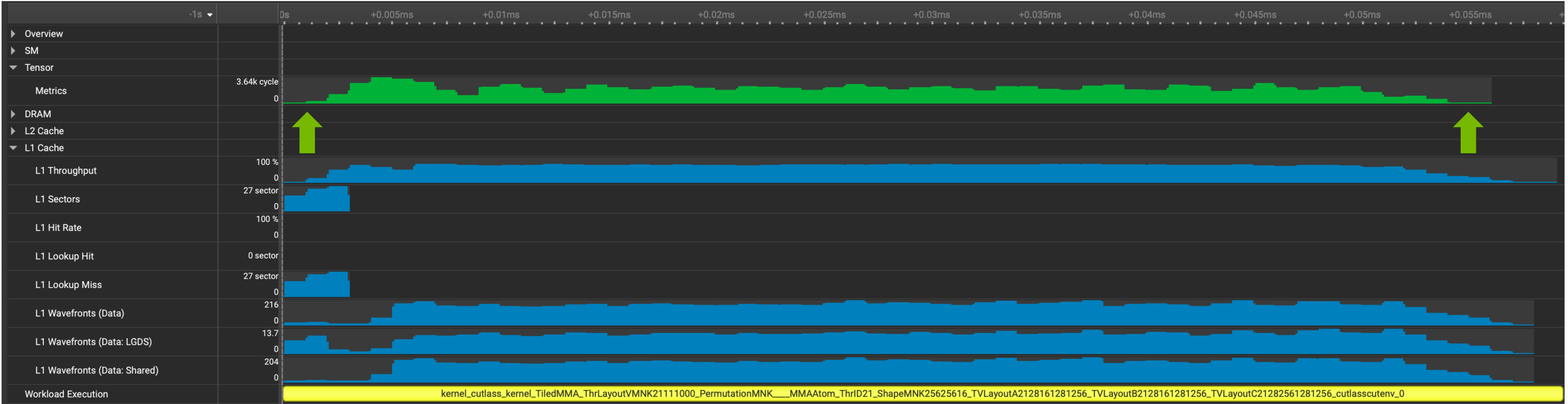
Nsight Compute views for multi-wave cases

Non-Persistent Kernel



All prologues and epilogues are exposed!

Persistent Kernel



Only the first prologue and the last epilogue are exposed

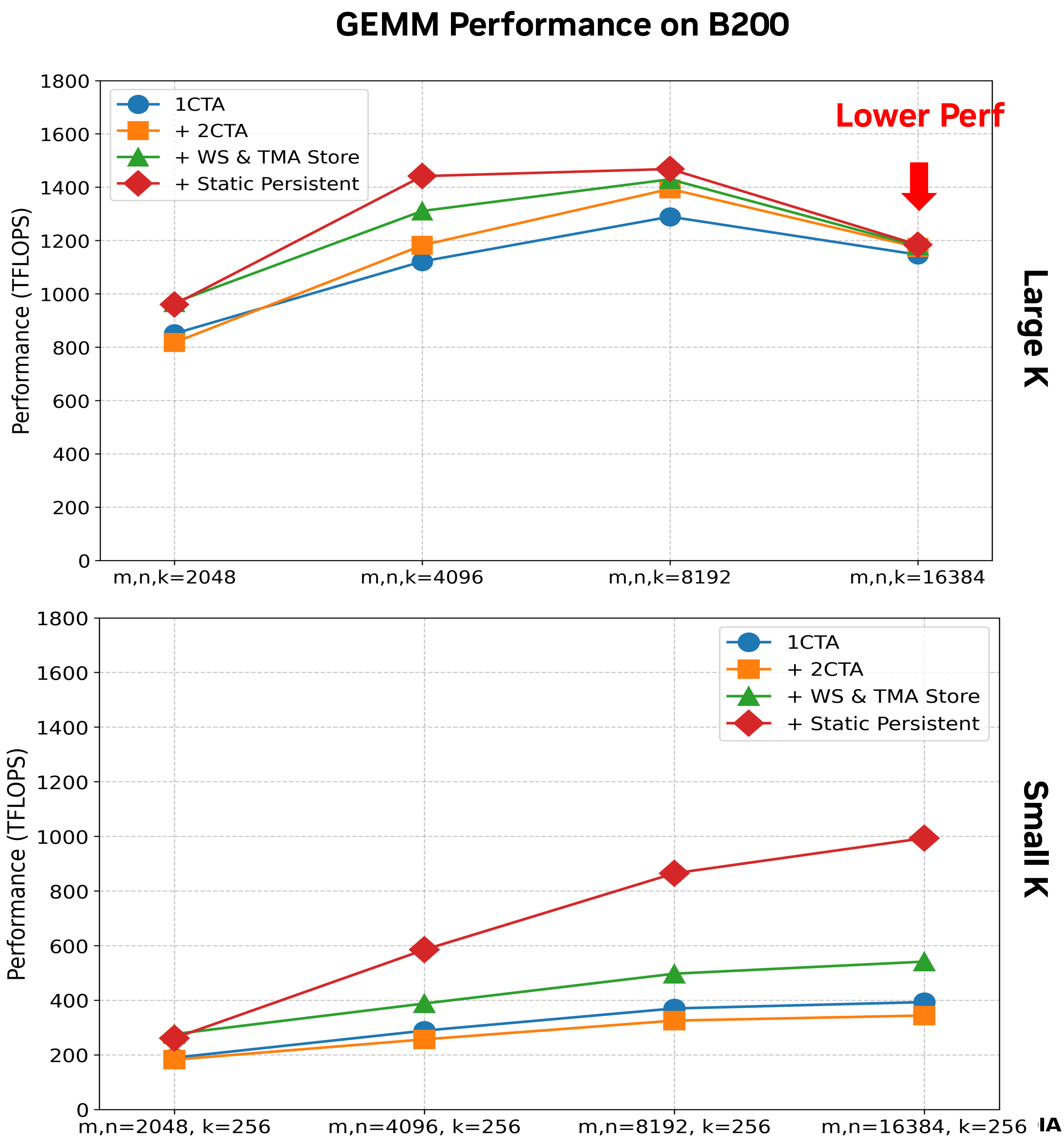
Tutorial 3: Performance Status

Static Persistent Kernel

Most cases benefits from Static Persistent Scheduler
We expect larger performance gains when the epilogue proportion is large, and vice versa

Why is there poor performance for 16Kx16Kx16K?

- Low L2 hit rate (32.9%) because of limited L2 size (e.g. 120 MB on B200)



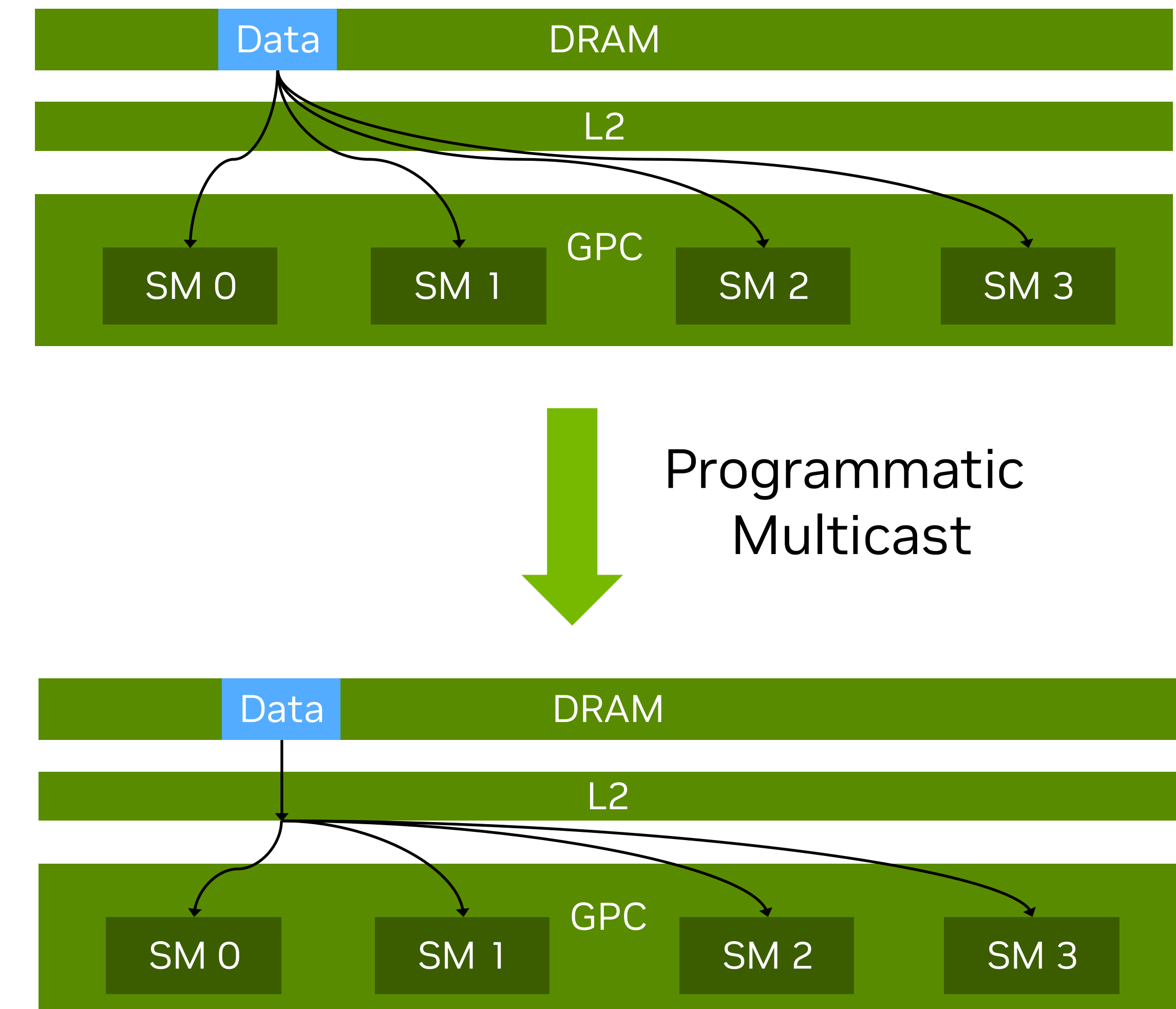
Tutorial 4: Preferred and Fallback Clusters

Larger cluster layout alleviates the situation:

- Large cluster layout → Programmatic Multicast → Reduced L2 traffic
- Large cluster layout → CTA Rasterization → High L2 Hit rate
- May result in poor SM occupancy due to quantization
 - Example: A 2x2 cluster on an 18 SM GPU would only utilize 16 SMs, leaving 2 SMs idle

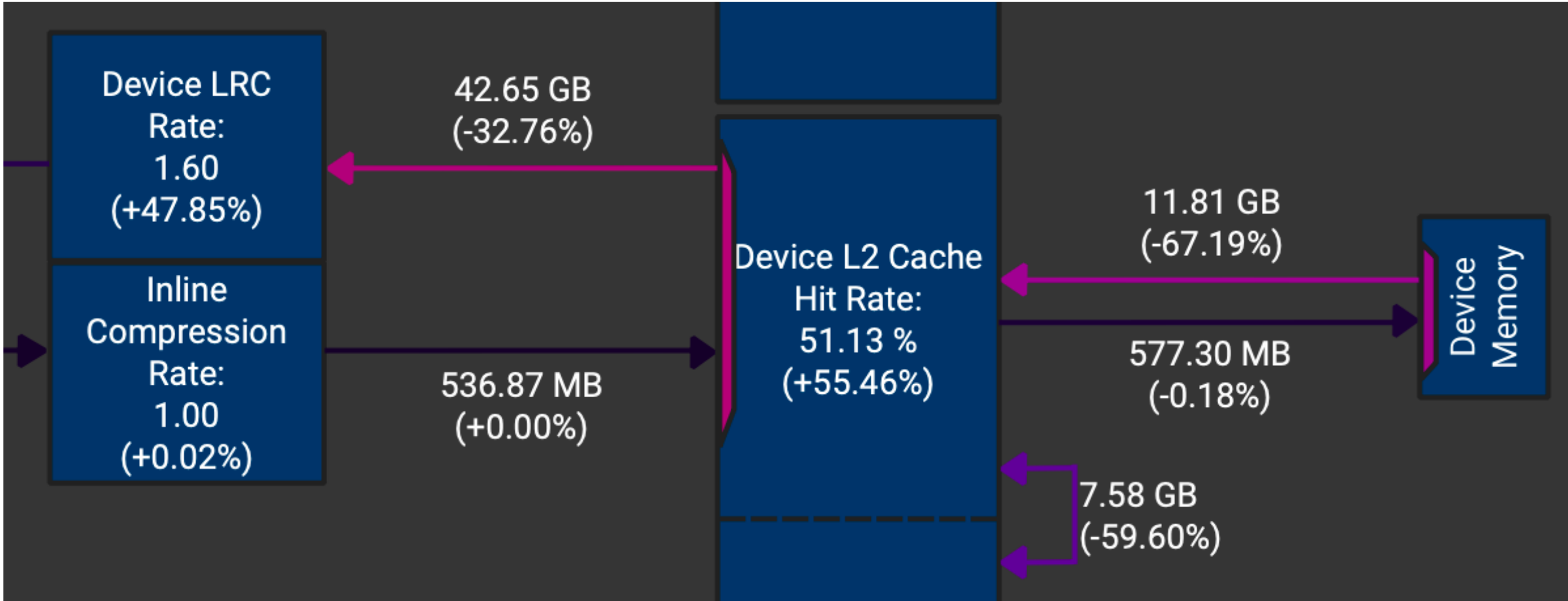
Blackwell supports specify two clusters:

- Preferred (large) Cluster: Achieve higher TMA multicast
- Fallback (small) Cluster: Enhance occupancy



Programmatic Multicast: One load to store into shared memory of multiple CTAs in the same cluster, to save L2 bandwidth

Tutorial 4: Performance Status

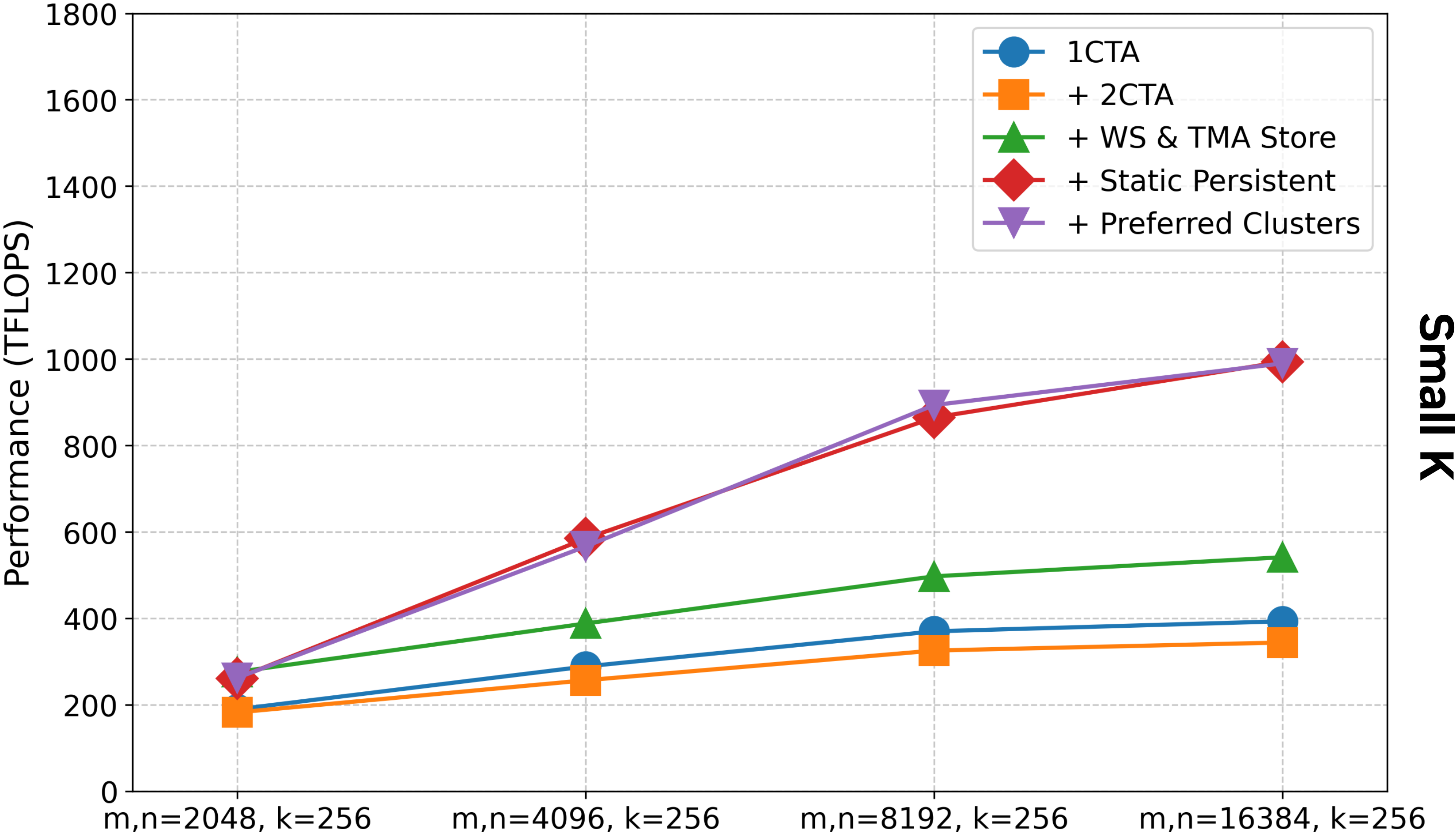
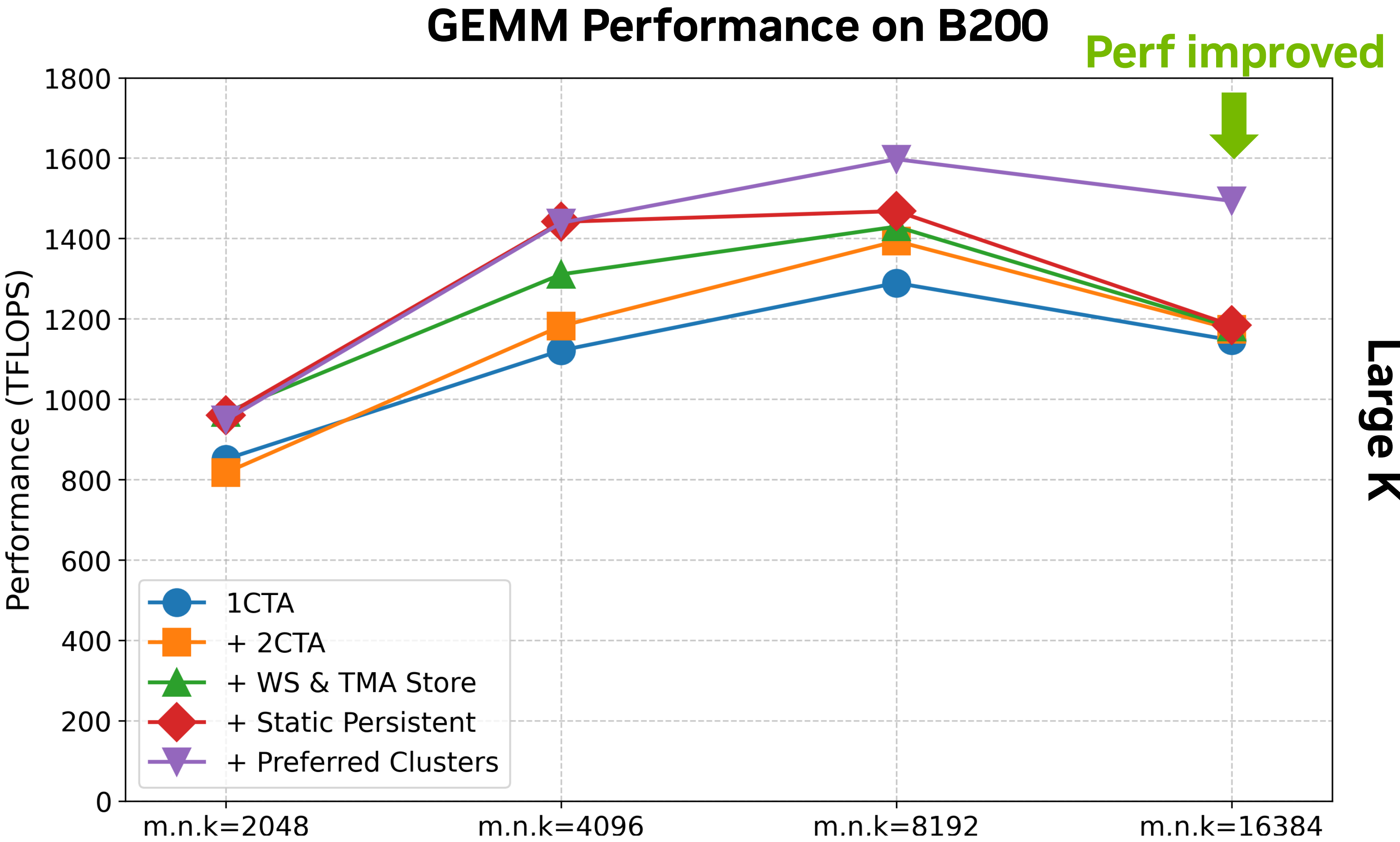


L2 hit rate before and after using Preferred Cluster
Large K Problem

MxNxK	2Kx2Kx2K	4Kx4Kx4K	8Kx8Kx8K	16Kx16Kx16K
w/o Preferred Cluster	97.25%	81.29%	46.17%	32.89%
w/ Preferred Cluster	97.07%	81.49%	73.63%	51.13%

Performance benefits of Preferred Cluster:

- Fully utilize the SMs while retaining the benefits of large cluster size
- Preferred and Fallback Cluster mainly helps the L2-bound cases



Tutorial GEMM Optimization Summary

- CUTLASS Python is a powerful tool for authoring performant kernels
- Optimization recap:
 - **2CTA + Multicast:** Reduced per-CTA SMEM allows deeper pipelining, L2 traffic is reduced due to TMA multicast
 - **Warp Specialization & Epilogue optimization:** Warp specialization provides better instruction mix and higher memory coalescing write
 - **Persistent Scheduler:** Persistent clusters hide prologue and epilogue costs among waves
 - **Preferred & Fallback Clusters:** Balance large and small cluster shapes cluster shape for higher TMA multicast and improved occupancy

Strategies Used	Cumulative Benefits	Relative Performance Change (M=N=8192, K={8192, 256})
Software-pipelined GEMM	Baseline implementation	1.00x, 1.00x
+ 2 CTA optimization	Improved latency hiding	1.08x, -1.10x
+ Warp-specialization and TMA store	Better epilogue phase overlap	1.11x, 1.34x
+ Static Scheduling persistency	Pipeline bubbles hidden among waves	1.14x, 2.34x
+ Preferred and fallback clusters	SMs occupancy improved	1.24, 2.41x

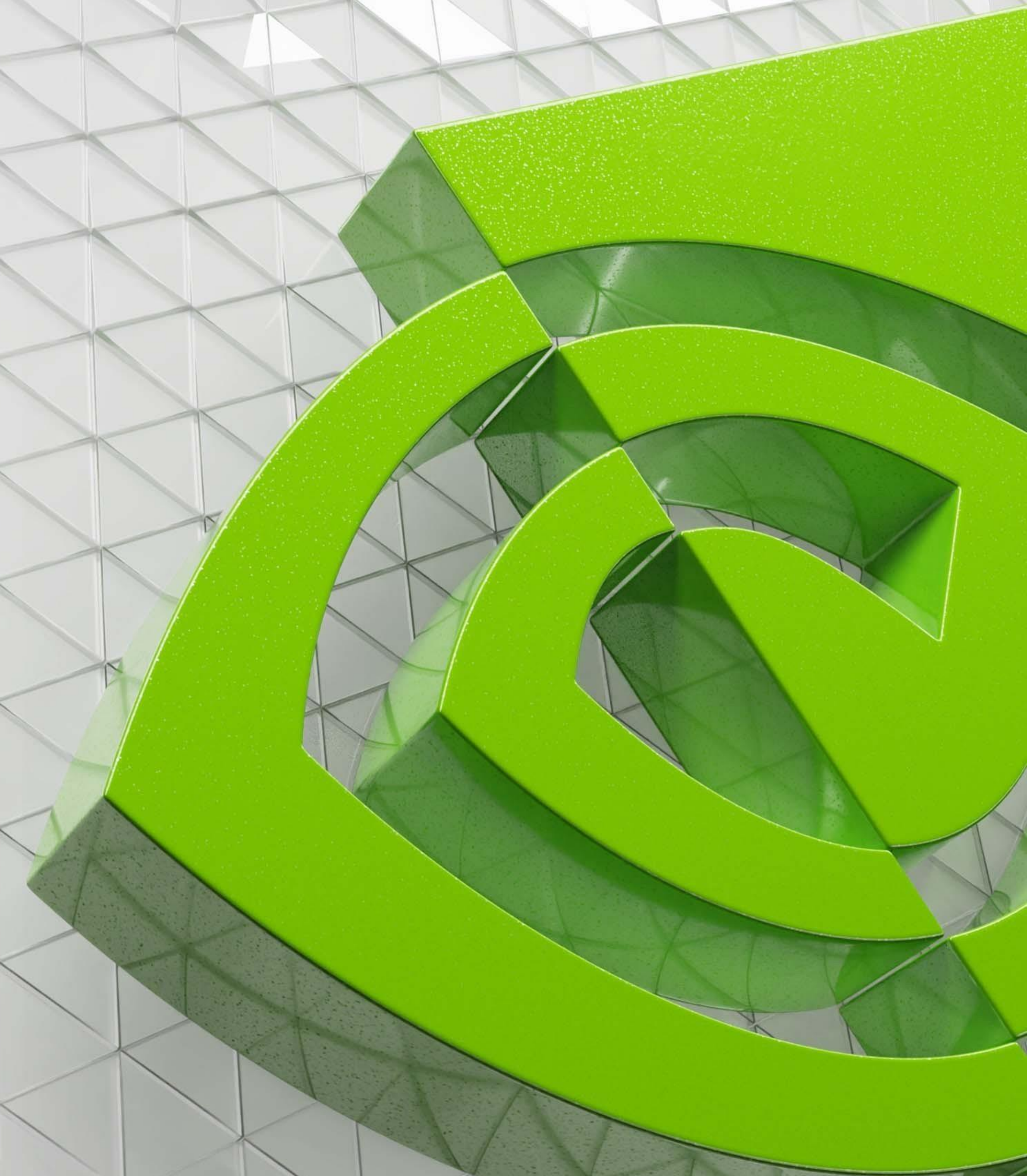
Agenda

Introduction

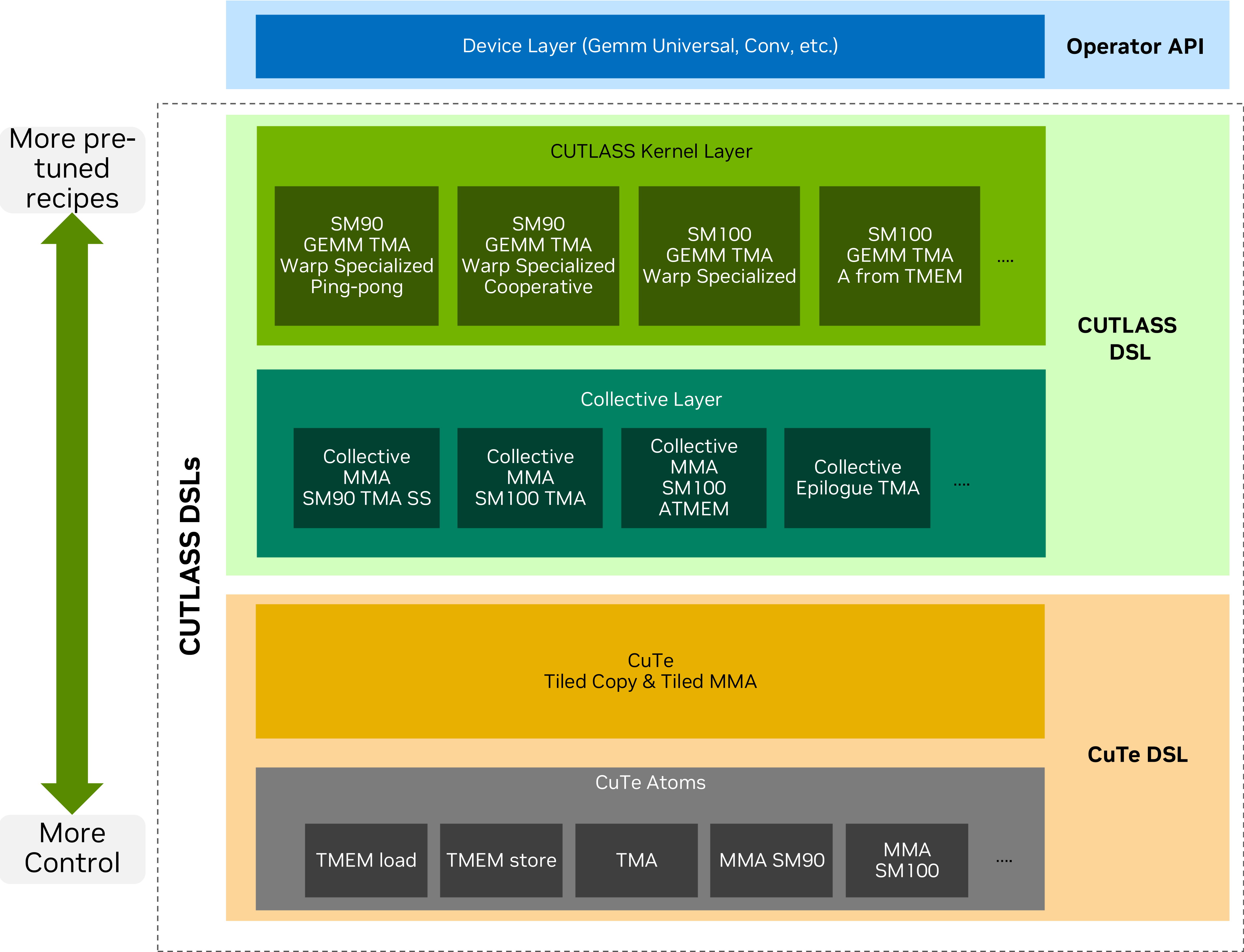
Progressive GEMM Optimization

CUTLASS DSLs

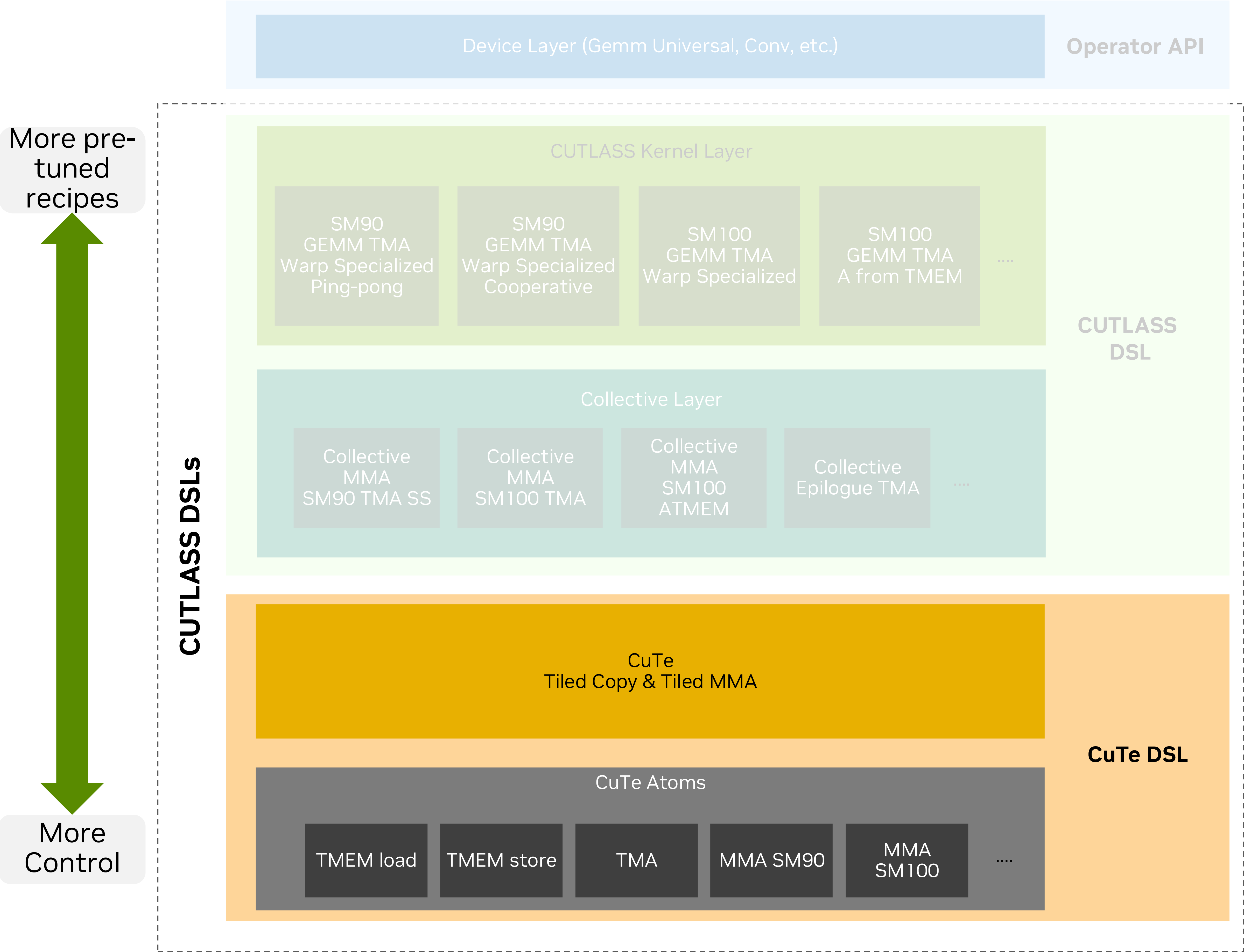
Python Operator API



CUTLASS Python Stack



CUTLASS Python Stack



New CuTe DSL Functionality

A thin abstraction layer on top of CuTe

Minimally abstract manual chores in CuTe

- Features that require global analyses, instead of local rewrites or helper functions
- Abstracts the CuTe programming model for convenience

Retain full inter-operability: No performance compromise. No representation gaps

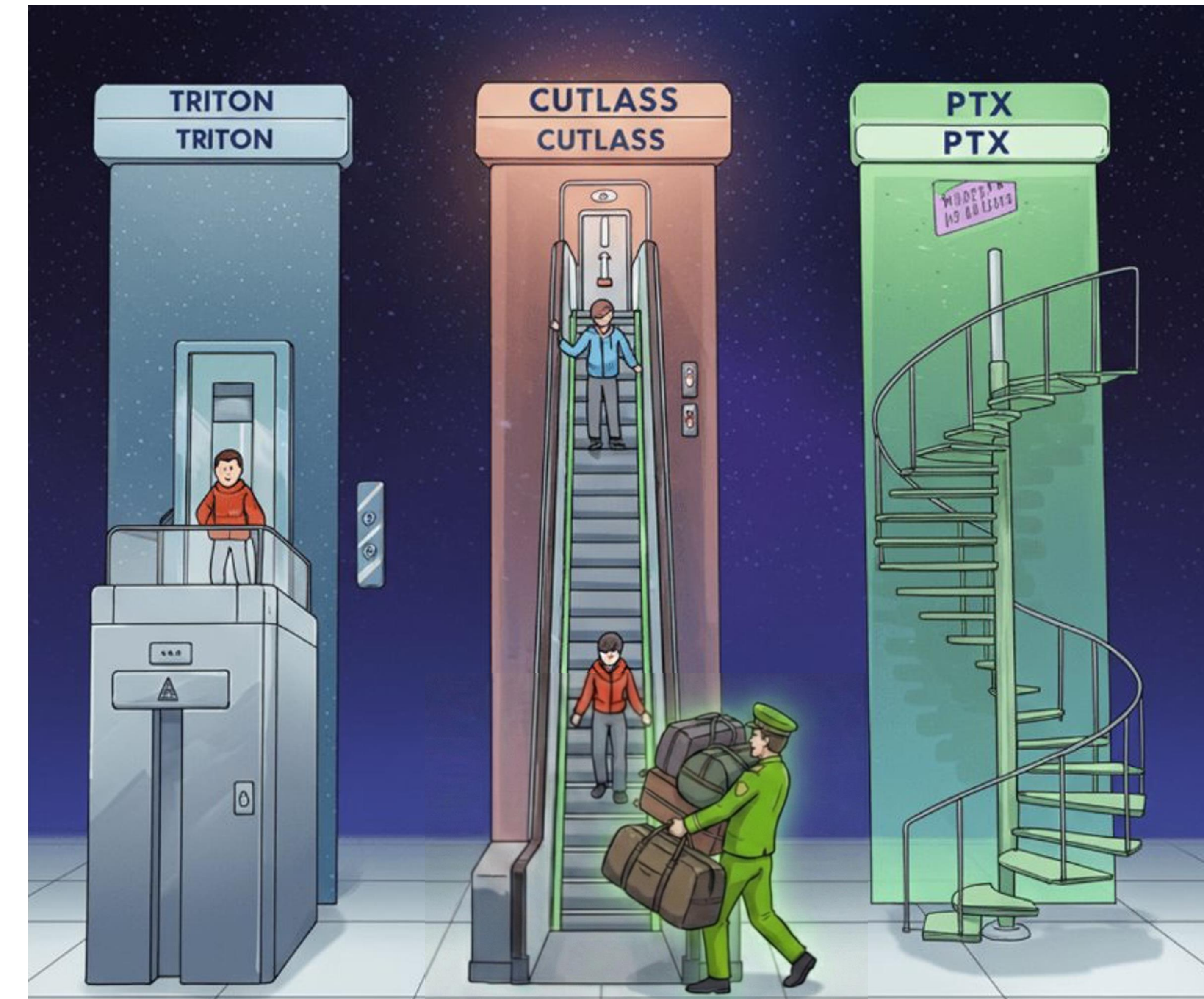
- All abstractions compose seamlessly with CuTe

Extensions through CuTe DSL under custom namespace and decorator

- Released in 4.4 as `@cute.experimental.jit` and `@cute.experimental.kernel`
- In 4.6, will just be `@cute.jit` and `@cute.kernel`

Available now:

<https://bit.ly/cute-dsl-extensions>



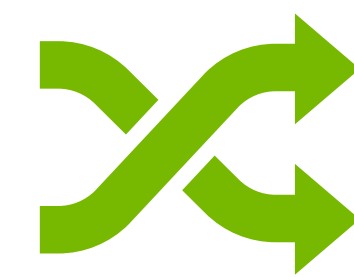
Offload heavy chores
Equal outcome
Opt-in convenience, bypass anytime

CuTe DSL Automates Chores

- Automate **error-prone, tedious** orchestration
- **Opt-in convenience**, bypass at any time for full control



Automatic TMA coord tensor transform,
TMA descriptor creation,
Outlining to host code



Automatic TMA desc update
on-device when kernel
ptr/shape/strides change



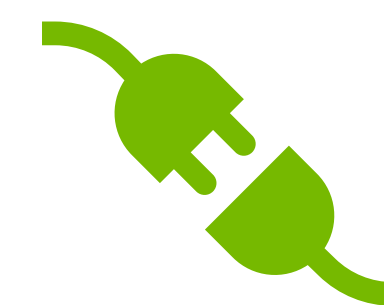
Automatic GMEM predication
based on dynamic shapes
input to the kernel



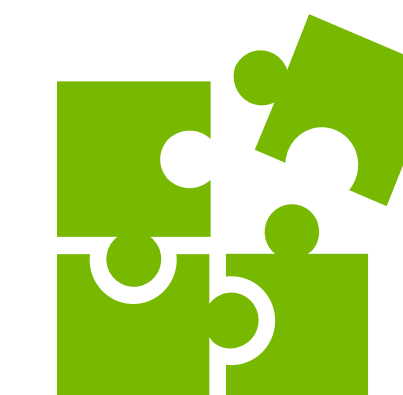
Automatic vectorization
for SIMT copies



Programmatic interface for
device workspace allocation
in addition to compiler-
managed workspace



Automatic host functions /
runtime hooks generation



Fragment-free programming
model for all MMA and copies

Example: Simpler TMA orchestration

CuTe DSL without extensions

```
# -- Host code -- Setup TMA load for GMEM tensor
tma_a = cpasync.make_tiled_tma_atom(
    a_op, a,
    self.a_smem_layout_staged,
    (self.mma_tiler[0] // atom_thr_size, self.mma_tiler[2]),
    internal_type=(
        cutlass.TFloat32 if a.element_type is cutlass.Float32 else None),
)
# -- Device code -- Takes TMA desc as input
@cute.kernel
def kernel(
    tma_a: cpasync.TmaInfo, ...
):

    # Prefetch TMA descriptor
    if warp_idx == tma_load_warp_id:
        cute.nvgpu.cpasync.prefetch_descriptor(tma_a.atom)

    # Tile & Partition TMA coord tensor, make fragments
    gA = cute.local_tile(mA_mkl, mma_tiler_mnk, mma_coord_mnk, proj=(1, None, 1))
    tAsA, tAgA = cpasync.tma_partition(
        tma_atom_a,
        block_in_cluster_coord_vmnk[2],
        a_cta_layout,
        cute.group_modes(sA, 0, 3),
        cute.group_modes(tCgA, 0, 3),
    )
    tAgA_slice = tAgA[(None, mma_tile_coord_mn1[0], None, mma_tile_coord_mn1[2])]

    # Issue TMA load
    cute.copy(
        tma_a.atom,
        tAgA_slice[(None, k_tile_idx)],
        tAsA[(None, handle.index)],
        tma_bar_ptr=handle.barrier,
        mcast_mask=None)
```

CuTe DSL with extensions

```
# -- Host code -- No setup needed!
```

No host TMA initialization

```
# -- Device code -- Takes GMEM tensor directly
```

```
@cute.experimental.kernel
def kernel(
    mA: cute.Tensor, ...
):
```

Kernel input is regular tensor

No TMA descriptor management

```
# Tile the GMEM tensor directly
```

```
gA = cute.zipped_divide(mA, tiler_mn)
gA_tile = gA[(None, None), (cta_m, cta_n, cta_l)]
gA_k = gA_tile[(None, None, k)]
```

No TMA coordinate tensors

```
# Issue TMA load
```

```
cute_ext.tma_load_multicast(
    gA_k,
    sA[(None, None, None, idx)],
    mbarrier,
)
```

Example: Automatic TMA Update in Grouped GEMM

CuTe DSL without extensions

```
# -- Host code -----
# Setup dummy TMA atom & tensor
tma_atom_a, tma_tensor_a = cute.nvgpu.make_tiled_tma_atom_A(...)
# Allocate GMEM workspace for TMA descs
tensormaps_workspace_buffer = ...

# -- Device code -----
@cute.kernel
def kernel(
    strides_abc: cute.Tensor,
    ptrs_abc: cute.Tensor,
    tma_atom_a: cute.CopyAtom,
    tma_atom_b: cute.CopyAtom,
    tma_atom_c: cute.CopyAtom,
    tensormaps_workspace: cute.Tensor,
):
    # Select workspace & copy TMA descs to workspace
    tensormap_manager = utils.TensorMapManager(
        self.tensormap_update_mode, GroupedGemmKernel.bytes_per_tensormap)
    tensormap_a_ptr = tensormap_manager.get_tensormap_ptr(
        tensormaps_workspace[(tensormap_workspace_idx, 0, None)].iterator)
    if cutlass.const_expr(self.delegate_tensormap_ab_init == False):
        tensormap_manager.init_tensormap_from_atom(
            tma_atom_a, tensormap_a_init_ptr, self.tma_warp_id)

    # Change/create tensor stride & ptr on the fly
    real_tensor_a = cute.make_tensor(ptrA, layoutA)

    # Ensure TMA initialization is complete before updating
    if not tensormap_init_done:
        tensormap_manager.fence_tensormap_initialization()
    # Update TMA descriptor
    tensormap_manager.update_tensormap(
        real_tensor_a, tma_atom_a, tensormap_a_ptr,
        self.tma_warp_id, tensormap_a_smem_ptr)
    # Ensure update is complete before using
    tensormap_manager.fence_tensormap_update(tensormap_a_ptr)

# FINALLY Issue TMA copy
cute.copy(...)
```

CuTe DSL with extensions

```
# -- Host code -----

# -- Device code -----
@cute.experimental.kernel
def kernel(
    strides_abc: cute.Tensor,
    ptrs_abc: cute.Tensor,
):

# Change/create tensor stride/ptr on the fly
mA = cute.make_tensor(ptrA, layoutA)

# Tile & issue TMA copy as usual
cute_exp.tma_load_multicast(...)
```

No host TMA initialization

Workspace auto managed

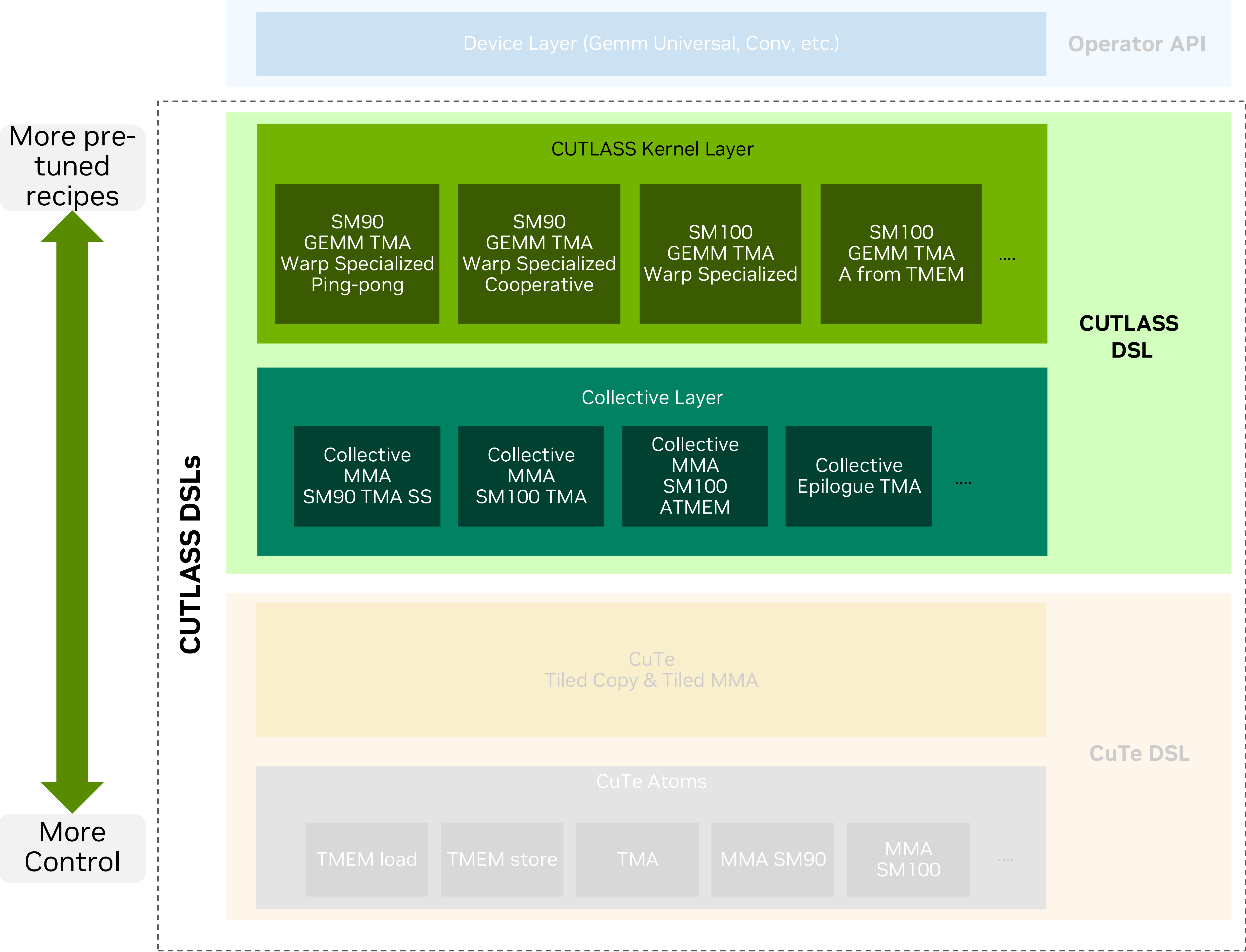
No dummy TMA atom

No TMA descriptor management

No TMA coordinate tensors

TMA update auto detected

CUTLASS Python Stack



CUTLASS DSL Preview

Streamlining Kernel Development

Goal: Specify algorithms in terms of tensors without requiring all layout details

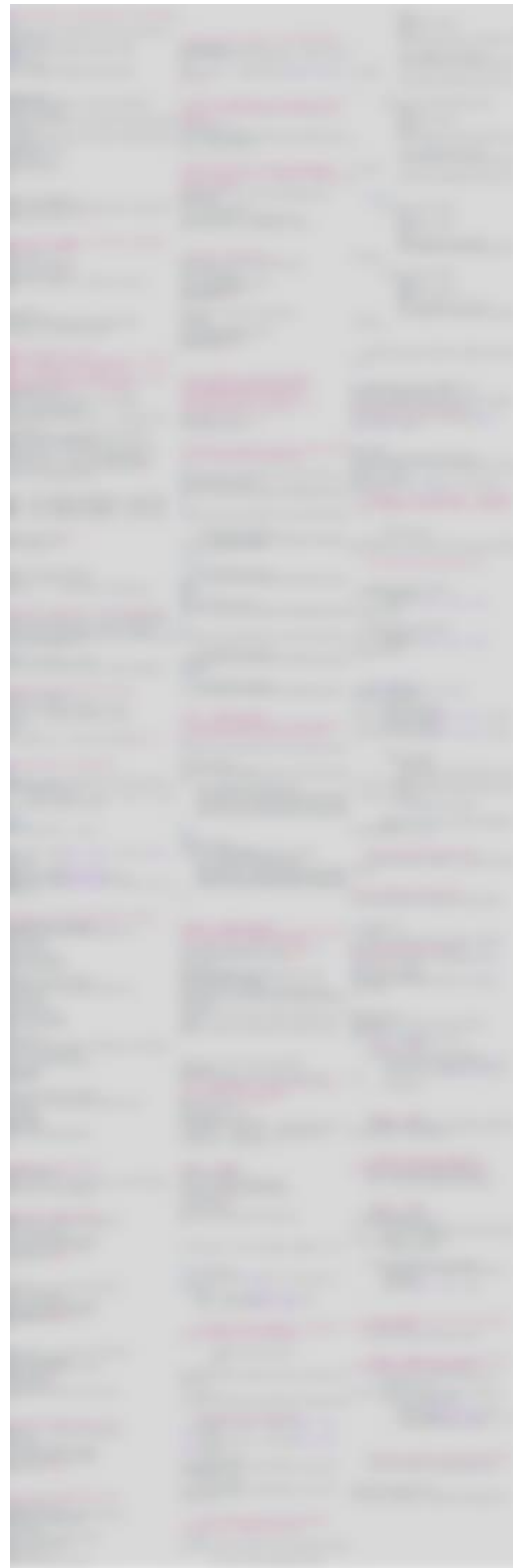
- Higher-level DSL, extending beyond Cute DSL
- Represent warp-specialized kernels with an explicit task graph
- Compiler becomes the CuTe expert enabling focus on high-level machine mapping

New division of labor:

- Programmer manages pipelines, agent structure, SMEM/TMEM layouts, and decides MMA and Copy atom
- Compiler constructs partitioning chain, manages buffers, and inserts synchronization

Coming later this year 😊

CuTe DSL GEMM Kernel



**386 lines
to 34 lines!**



CUTLASS DSL GEMM Kernel

```
with example_dsl.task_graph() as graph:
    tma_dep = graph.dependency([smem_a_ty, smem_b_ty])
    mma_dep = graph.dependency([tmem_acc_ty])

@graph.agent(
    "Thread", 0, 32, # MMA agent (threads 0-31):
    produces=[mma_dep], # consumes A,B, produces accumulator
    consumes=[tma_dep],
)
def mma_agent(acc_queue, ab_consumer):
    accum_init = example_dsl.zeros(tmem_acc_ty)

    for k, acc, result in cutlass.for_generate(
        0, tile_count_K, 1, [accum_init]
    ):
        A, B = example_dsl.fifo_pop(ab_consumer, [smem_a_ty, smem_b_ty])
        acc = example_dsl.dot(
            A, B, acc,
            op_type=OperationTypeEnum.SM100_MMA_1SM_SS,
            tiled_mma=tiled_mma,
        )
        cutlass.yield_out([acc])
    acc = result
    example_dsl.fifo_push(acc_queue, acc)

@graph.agent(
    "Thread", 32, 32, # TMA agent (threads 32-63)
    produces=[tma_dep], # loads A,B tiles GMEM->SMEM
)
def tma_agent(ab_queue):
    for k in cutlass.range(0, tile_count_K, 1, unroll=1):
        A = example_dsl.tma_load(gA_tile[None, None, k], smem_a_ty, v_map=a_v_map)
        B = example_dsl.tma_load(gB_tile[None, None, k], smem_b_ty, v_map=b_v_map)

        example_dsl.fifo_push(ab_queue, A, B)

@graph.agent(
    "Thread", 128, 128, # Epilogue agent (threads 128-255)
    consumes=[mma_dep], # TMEM->RMEM->GMEM
)
def epilogue_agent(acc_consumer):
    D_acc = example_dsl.fifo_pop(acc_consumer, [tmem_acc_ty])[0]
    D_rmem = example_dsl.copy(
        D_acc, rmem_d_ty,
        op_type=OperationTypeEnum.SM100_COPY_T2R,
        tiled_copy=tiled_copy_t2r,
    )

    example_dsl.store(
        gD_tile.value, D_rmem,
        op_type=OperationTypeEnum.STG,
        tiled_copy=tiled_copy_r2g,
    )
```

Agenda

Introduction

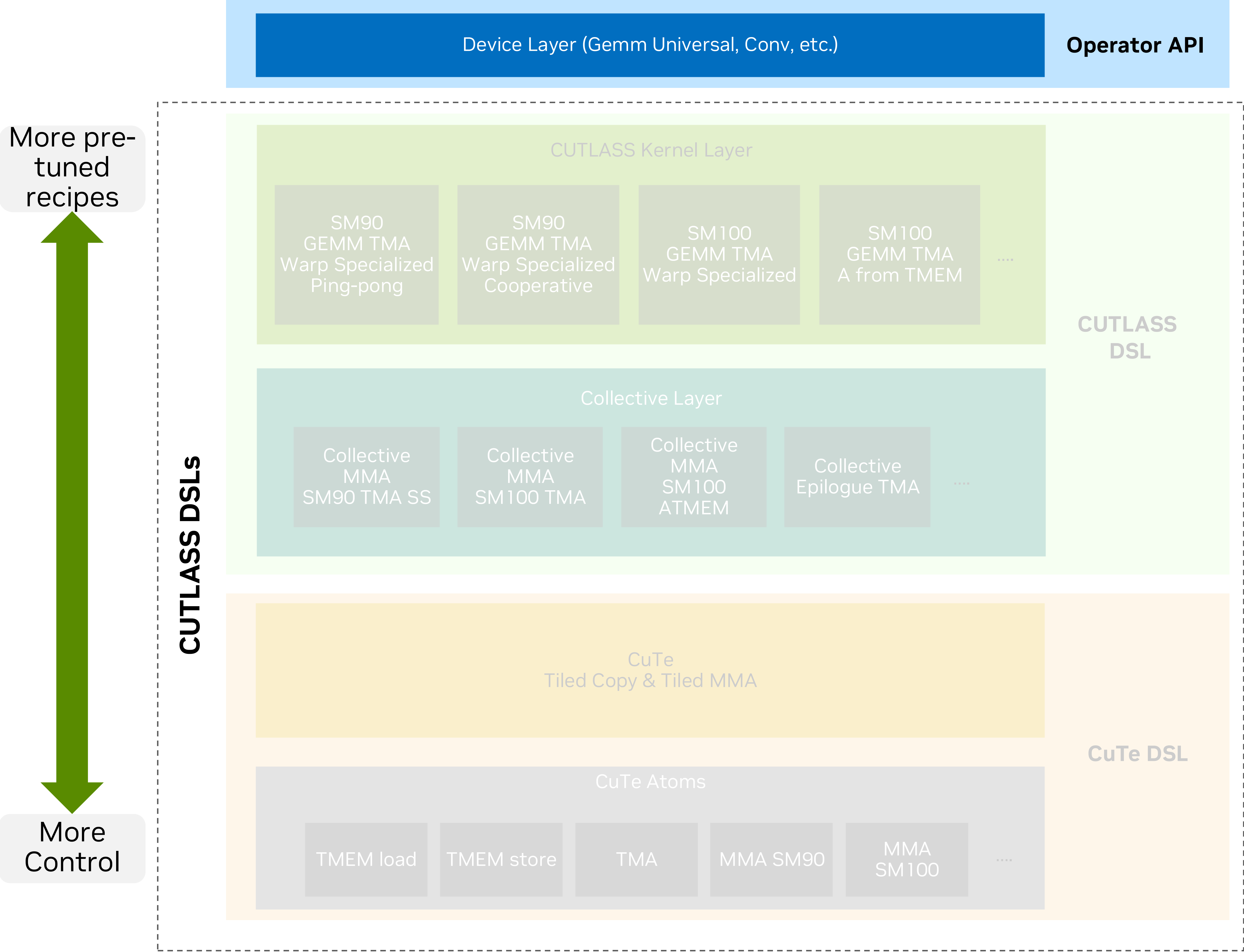
Progressive GEMM Optimization

CUTLASS DSLs

Python Operator API



CUTLASS Python Stack



Operator API

Coming Soon!

Universal Interface and Builders for GEMMs and Other Critical Operations

DSLs:
Write performant kernels



Operator API:
Integrate written kernels into
production libraries

- Unified interface across different kernels & DSLs to discover, inspect, execute kernels
- First-class support for native PyTorch / JAX tensors

```
A, B, out = (torch.randn(128, 128, device="cuda", dtype=torch.float16) for _ in range(3))

# Describe operation with kernel-agnostic arguments
args = operator_api.GemmArguments(A, B, out, accumulator_type=torch.float32)

# Find supported kernels from different implementations across DSLs (e.g., CuTe DSL, cuTile)
kernels = operator_api.get_kernels(args)

# JIT compile and run one of the kernels
kernels[0].run(args)
```



Unified Interface
Across Kernels & DSLs



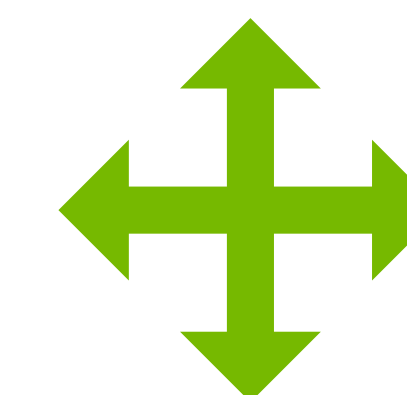
Perf Parity With
Cute DSL



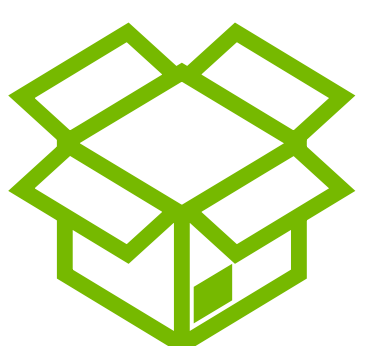
Discover 1000s
of Kernels



Custom Epilogues
From Python Functions



Wide Coverage Of
Prod-grade Kernels



Bring Your
Own Kernels

Operator API

Universal Interface and Builders for GEMMs and Other Critical Operations

Stable, portable integration layer for your library, while underlying Kernels & DSLs evolve

Simplify your integration code

- Uniform interface across kernels and DSLs
- Eliminate glue code to convert PyTorch / JAX → CuTe Tensors, find & inspect kernels, manage calling conventions

New kernels accessible as they are released, with no code change

No performance drop, negligible runtime overhead



Unified Interface
Across Kernels & DSLs



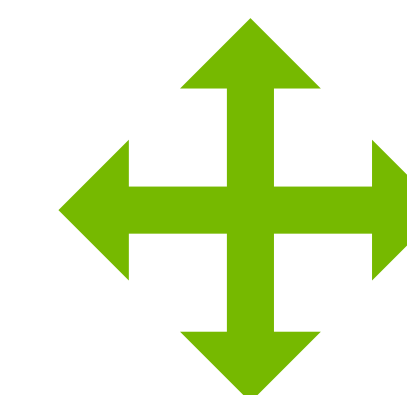
Perf Parity With
Cute DSL



Discover 1000s
Of Kernels



Custom Epilogues
From Python Functions



Wide Coverage Of
Prod-grade Kernels



Bring Your
Own Kernels

Operator API

```
using CollectiveEpilogue = typename
epilogue::collective::CollectiveBuilder<
    arch::Sm100, arch::OpClassTensorOp,
    TileShapeMNK, ClusterShapeMNK,
    epilogue::collective::EpilogueTileAuto,
    ElementAcc, ElementCompute,
    ElementC, LayoutC, AlignmentC,
    ElementD, LayoutD, AlignmentD,
    epilogue::collective::EpilogueScheduleAuto
>::CollectiveOp;
using CollectiveMainloop = typename
gemm::collective::CollectiveBuilder<
    arch::Sm100, arch::OpClassTensorOp,
    ElementA, LayoutA, AlignmentA,
    ElementB, LayoutB, AlignmentB,
    ElementAcc,
    TileShapeMNK, ClusterShapeMNK,
    gemm::collective::StageCountAutoCarveout<carveout>,
    gemm::collective::KernelScheduleAuto
>::CollectiveOp;
using GemmKernel = gemm::kernel::GemmUniversal<
    Shape<int, int, int, int>,
    CollectiveMainloop,
    CollectiveEpilogue
>;
using Gemm = GemmUniversalAdapter<GemmKernel>;
```

CUTLASS 3.x

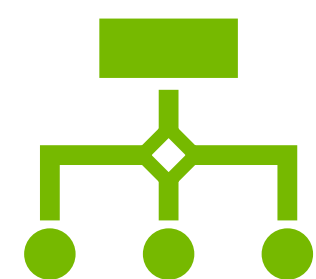


```
args = operator_api.GemmArguments(
    A, B, out,
    accumulator_type=torch.float32)

kernels = operator_api.get_kernels(args)
```

Operator API

- Query kernels by required functionality, not template params
- Opt-in advanced filtering available. Only specify what you need
- Coming Soon:
 - Heuristics for GEMM kernel selection
- **124k kernel configs** and counting...



Unified Interface
Across Kernels & DSLs



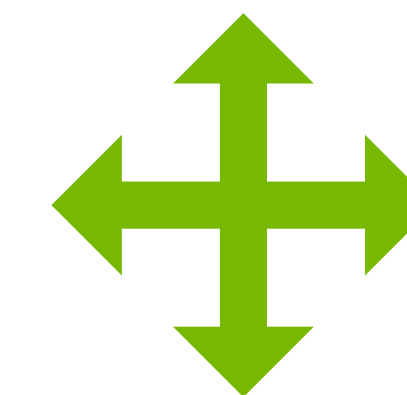
Perf Parity With
Cute DSL



Discover 1000s
Of Kernels



Custom Epilogues
From Python Functions



Wide Coverage Of
Prod-grade Kernels



Bring Your
Own Kernels

Operator API

Universal Interface and Builders for GEMMs and Other Critical Operations

```
# Create custom epilogue function
def my_epilogue(accum, alpha, beta, C):
    D = (accum * alpha) + (C * beta)
    return D

# Operator API traces the function
C = torch.empty(...)
epi_args = operator_api.EpilogueArguments(my_epilogue,
                                          alpha=1.5, beta=2.5, C=C, D=out)

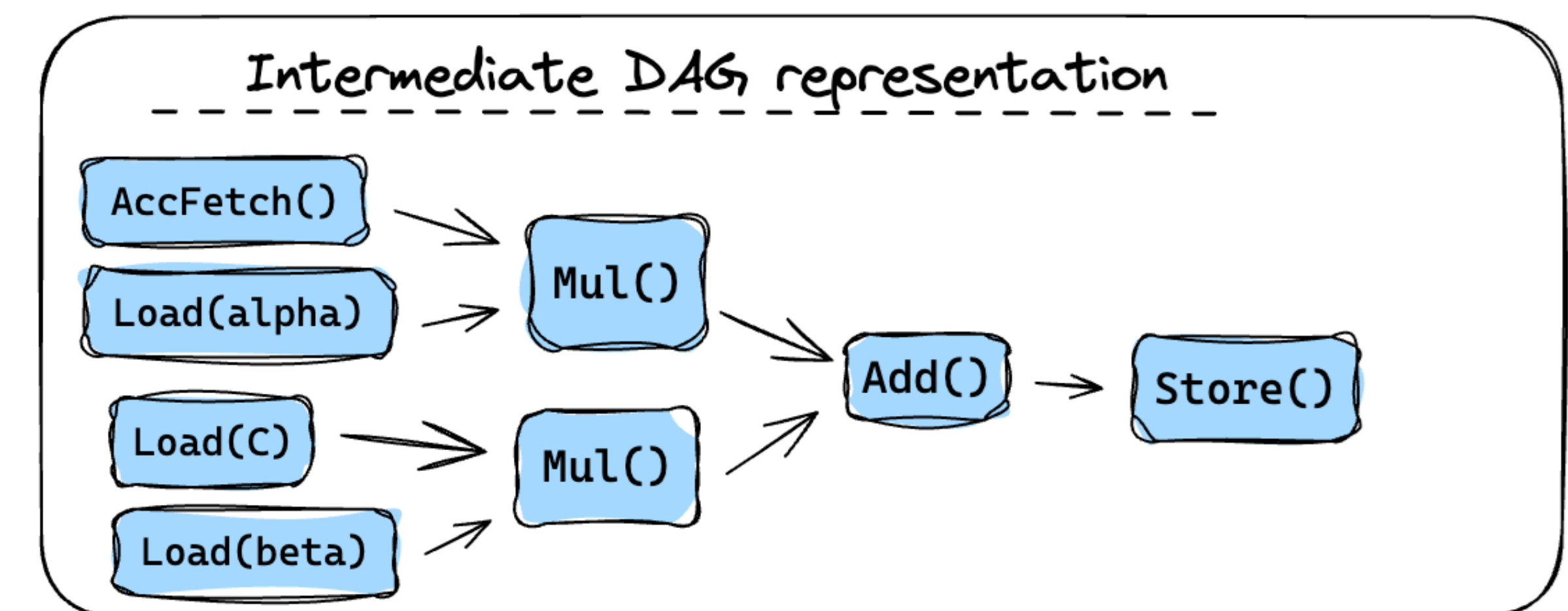
# Describe operation with kernel-agnostic arguments
args = operator_api.GemmArguments(A, B, out, accumulator_type=torch.float32,
                                  epilogue=epi_args)

# Find supported kernels from different implementations across DSLs
kernels = operator_api.get_kernels(args)

# JIT compile & run
kernels[0].run(args)
```

```
def epi(accum, alpha, beta, C):
    D = (accum * alpha) + (beta * C)
    return D
```

```
GemmArguments(...,
               epilogue=EpilogueArguments(
                   epi, alpha=1.5, beta=2.5, C=C, D=D))
```



Unified Interface
Across Kernels & DSLs



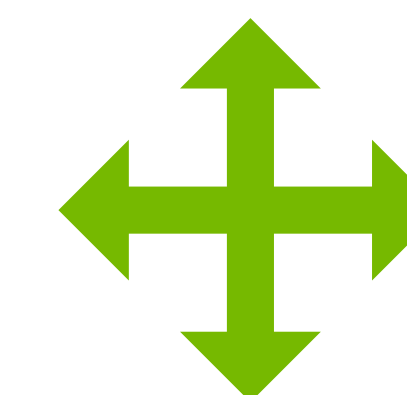
Perf Parity With
Cute DSL



Discover 1000s
Of Kernels



Custom Epilogues
From Python Functions



Wide Coverage Of
Prod-grade Kernels



Bring Your
Own Kernels

Operator API

Universal Interface and Builders for GEMMs and Other Critical Operations

Wide coverage of functionality, configs, and optimizations
Home for robustly-tested, optimized DSL kernels beyond examples

Initial release, available soon:

- CUTLASS DSLs & cuTile
 - Dense GEMMs (F32, F16, BF16, INT8)
 - Custom epilogue fusions (e.g., aux load/store, elementwise ops)
 - Block-scaled GEMM (MXFP8)
 - Grouped GEMM
 - **124k kernel configs** and counting...
- Bring Your Own Kernels
- CUDA Graphs
- PyTorch & JAX support
- Being integrated as a backend to **PyTorch Inductor**

Planned future releases:

- AOT support
- Kernel coverage
 - NVFP4
 - Low latency GEMMs
 - Sparsity
 - Blackwell Ultra optimizations



Unified Interface
Across Kernels & DSLs



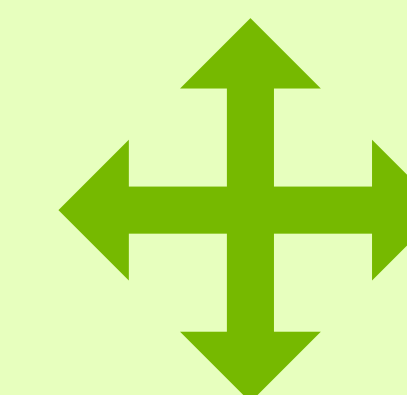
Perf Parity With
Cute DSL



Discover 1000s
Of Kernels



Custom Epilogues
From Python Functions



Wide Coverage Of
Prod-grade Kernels



Bring Your
Own Kernels

Thank You!

